"Advanced Statistics for Physics Analysis"
Final project report
ARPAV temperature trends in Veneto region

Kevin Brugnera Marco Lorenzato Libero Pollini

June 16, 2026

Introduction

This study investigates whether systematic temperature trends are detectable at selected meteorological stations in the Veneto region (North-Eastern Italy), over a 31-year span (1994-2024), with 30 years being considered the minimum amount of time for significant statistical inference in Climatology[1]. We analyze open data from the ARPAV archive[2], focusing on three stations at far-apart locations: Monte Summano (VI, on a mountain at around 600 m a.s.l.), Cittadella (PD), and Oderzo (TV) (both in the Veneto plain). Two Bayesian regression models are considered: a constant model (Model 1) and a linear trend model (Model 2):

$$M_1: \quad T_{s,t} = T_s^0 + \epsilon_{s,t}, \qquad \epsilon_{s,t} \sim \mathcal{N}(0, \sigma_s^2) \tag{1}$$

$$M_2: \quad T_{s,t} = T_s^0 + \dot{T}_s\, t + \epsilon_{s,t}, \qquad \epsilon_{s,t} \sim \mathcal{N}(0, \sigma_s^2), \tag{2}$$

where s indexes the station and t is time (at one-year intervals). Independent (and weakly informative) priors are used:

$$T_s^0 \sim \mathcal{N}(0, 20), \tag{3}$$

$$\dot{T}_s \sim \mathcal{N}(0, 20), \tag{4}$$

$$\sigma_s \sim \mathcal{N}(0, 10). \tag{5}$$

(normal distributions parametrized by mean and standard deviation). The following analysis comprises: (1) plots of monthly and annual averages of minimum, mean, and maximum temperature; (2) estimation of all station-specific parameters with credible intervals (CIs) via Monte Carlo Markov Chains (MCMC); (3) comparison of the constant and linear-trend models; and (4) 10-year posterior predictions for annual mean temperature for each station and model.

1 Data visualization

Below (Figure 1) are the plots of annual averages of minimum, mean, and maximum temperatures at each station. Aside from seasonal variations, we get the visual impression of slightly increasing temperatures over the years. This trend seems more pronounced in the minimum and mean temperatures, whereas the maximum temperature remains relatively constant over the years. Note that for the station in Oderzo, data for the last year of the series (2024) is only available for the month of January; therefore, we decide to omit it from the analysis.

[1] World Meteorological Organization (WMO). (2017). WMO Guidelines on the Calculation of Climate Normals (WMO-No. 1203).

[2] ARPAV, *Principali variabili meteorologiche*, available at `https://www.arpa.veneto.it/dati-ambientali/open-data/clima/principali-variabili-meteorologiche`

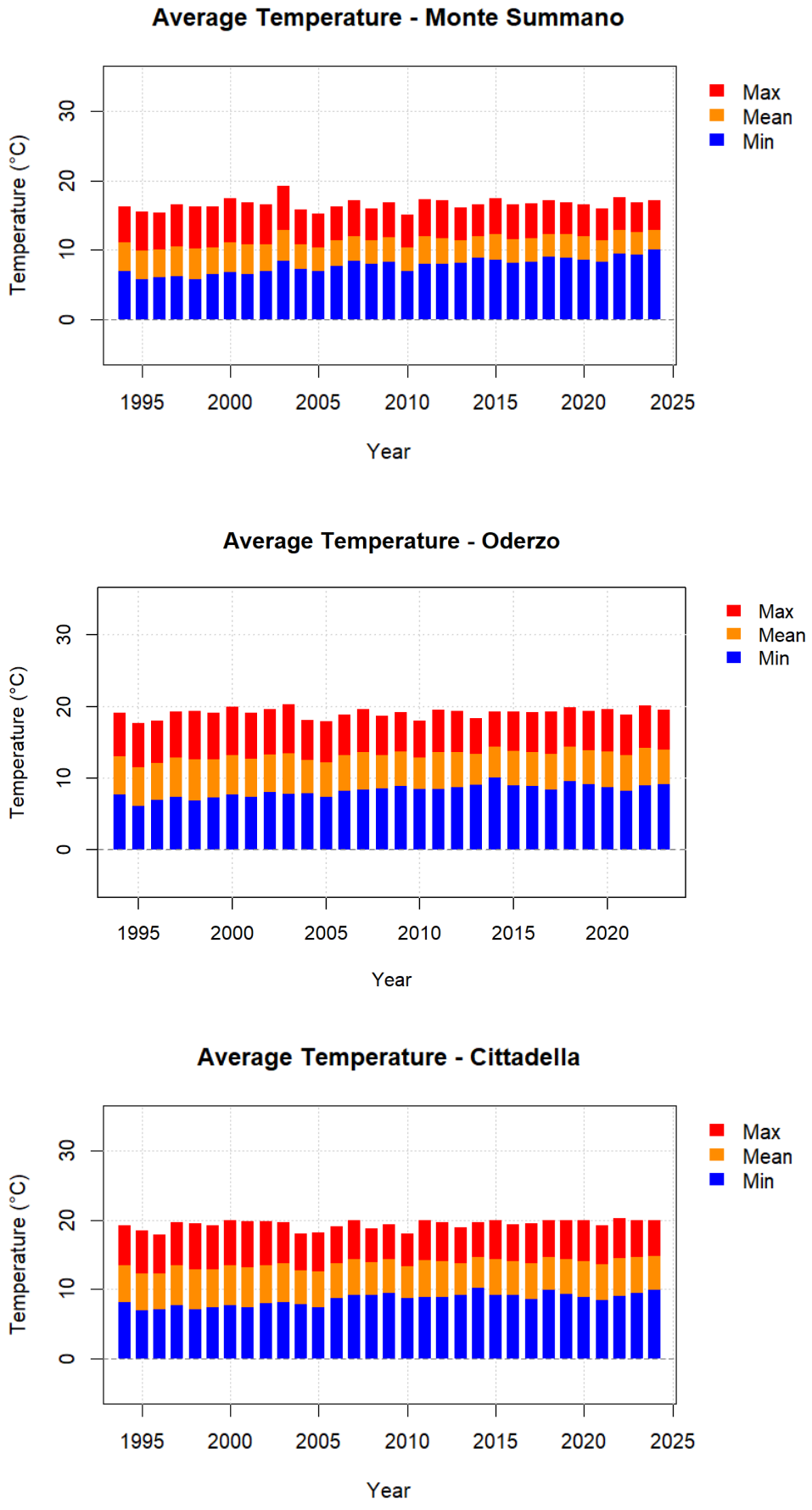


Figure 1: *Annual average temperatures for the three analysis locations.*

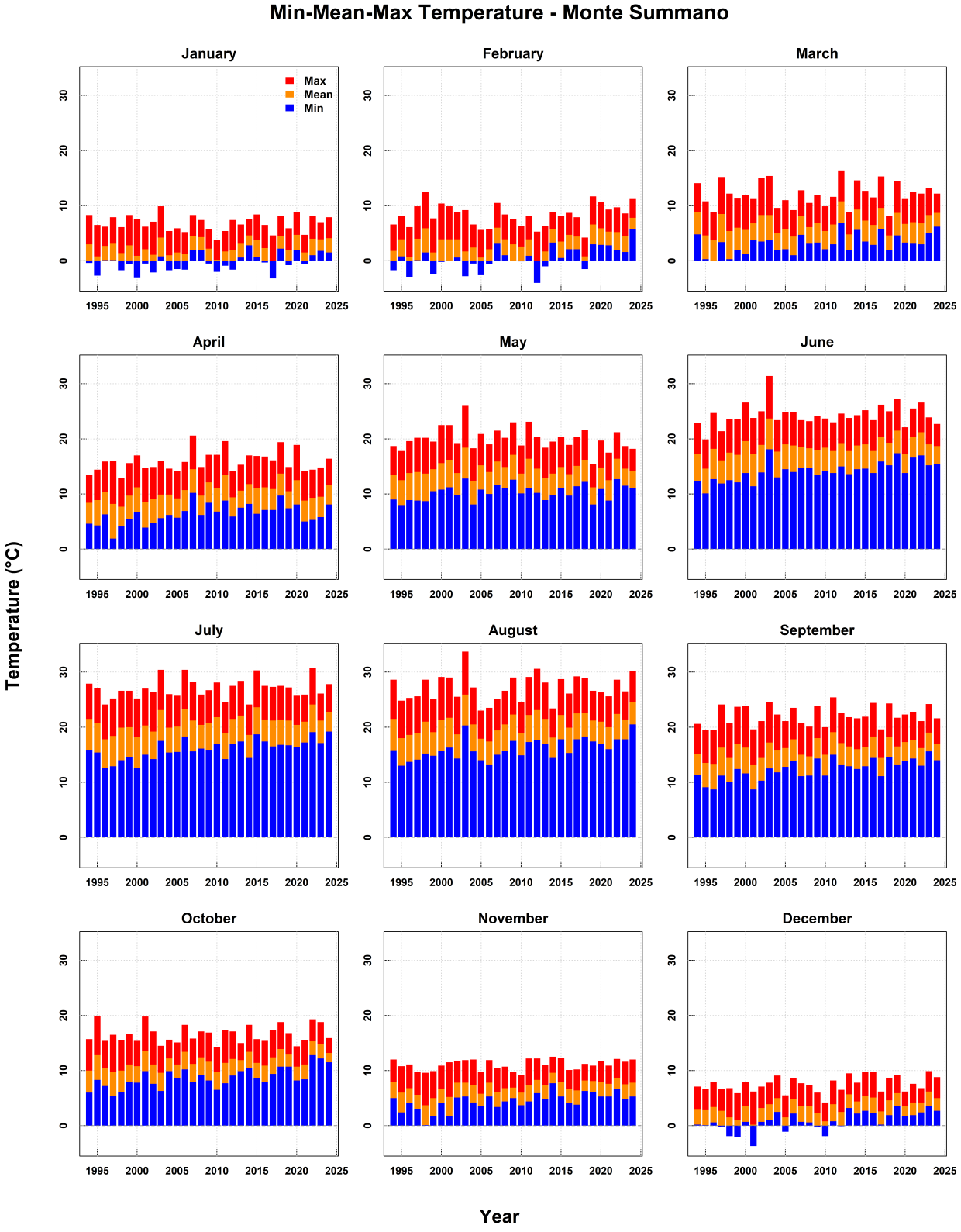


Figure 2: *Bar plot of the monthly minimum, mean, and maximum temperatures over the years, as reported by ARPAV for the Monte Summano station.*

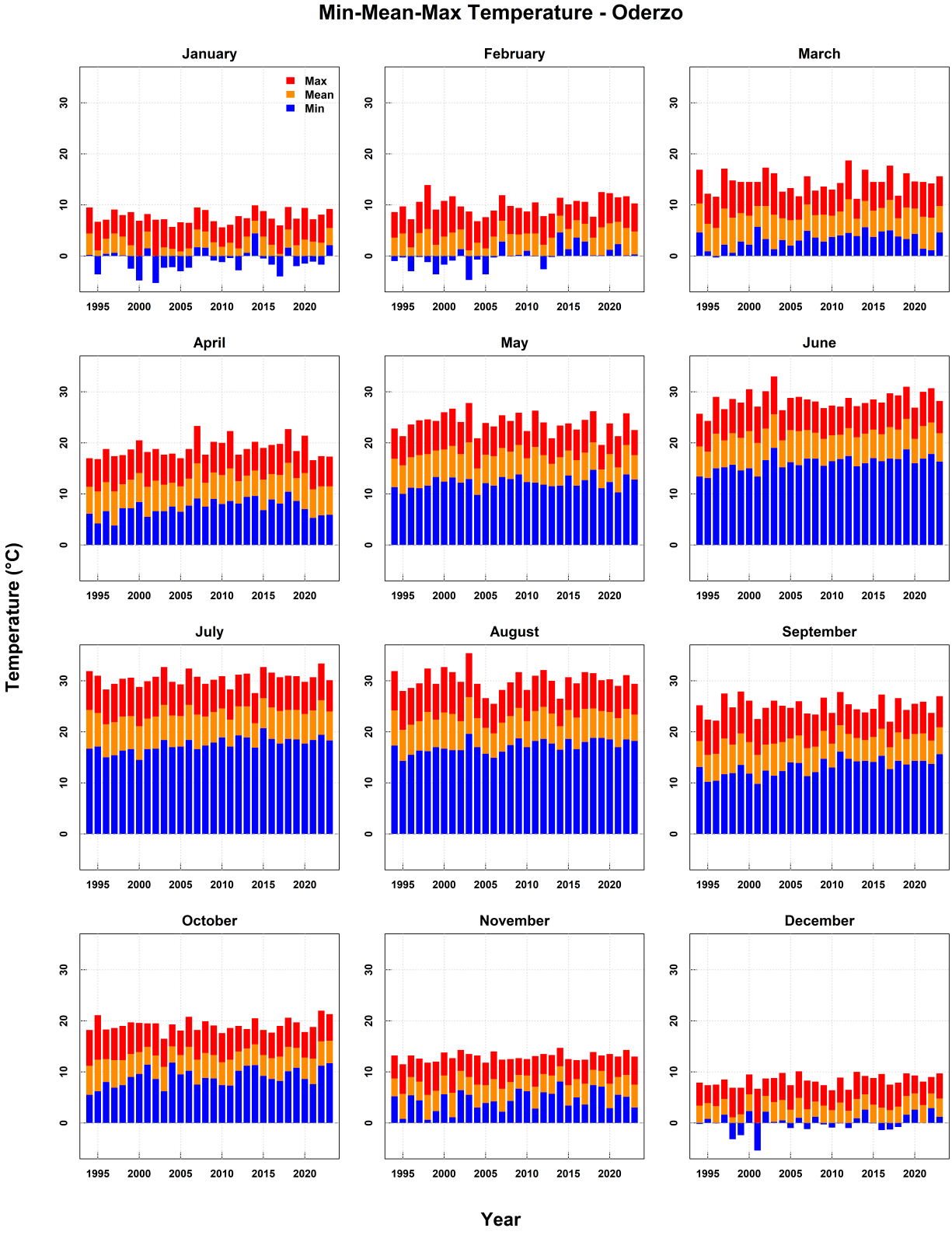


Figure 3: *Bar plot of the monthly minimum, mean, and maximum temperatures over the years, as reported by ARPAV for the Oderzo station.*

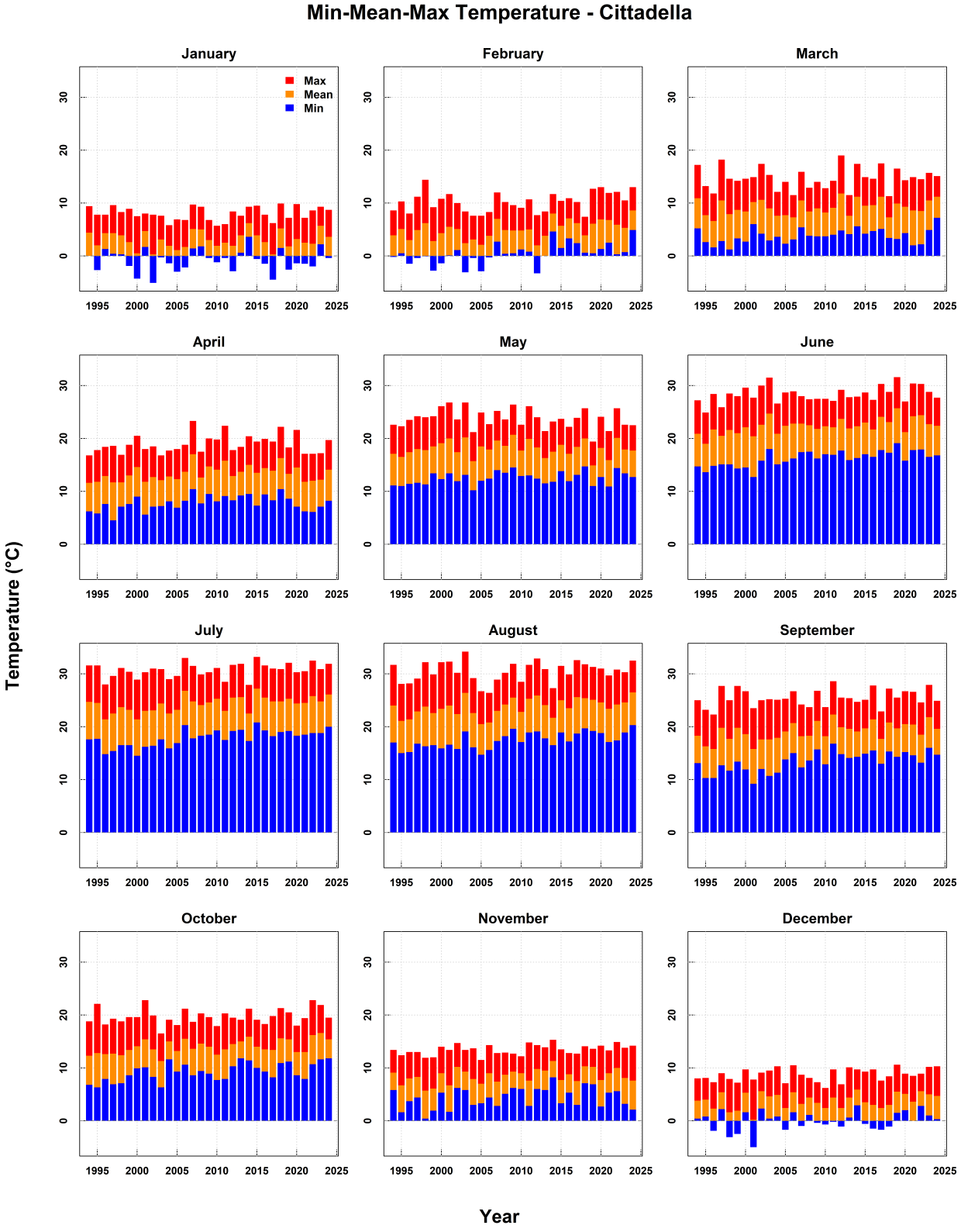


Figure 4: *Bar plot of the monthly minimum, mean, and maximum temperatures over the years, as reported by ARPAV for the Cittadella station.*

2 Parameter inference

We fit the Bayesian regression models mentioned above using the R library `Stan`[3], which uses Hamiltonian Monte Carlo (HMC) Markov Chains (MCMC), and explore its diagnostics. Below is the `.stan` file code which we used to build the linear model:

```
data {
int<lower=1> N;
vector[N] T;
vector[N] time;
}

parameters {
real T0;
real T_dot;
real<lower=0> sigma;
}

model {
T0 ~ normal(0, 20);
T_dot ~ normal(0, 20);
sigma ~ normal(0,10);

T ~ normal(T0 + T_dot * time, sigma);
}

generated quantities {
vector[N] log_lik;
for (i in 1:N)
log_lik[i] = normal_lpdf(T[i] | T0 + T_dot * time[i], sigma);
}
```

MCMC fitting and diagnostic

First, we standardize the t variable beforehand, to reduce posterior correlations between parameters and to make the prior scale invariant. For ease of computation, we decide to approximate the standard deviation to the nearest integer (10 for Monte Summano and Cittadella station, 9 for Oderzo). With the linear model, for each station we then run 4 parallel MCM Chains, each of 2000 iterations ("length"), preceded by a burn-in period of 1000 iterations. (Execution takes only a few seconds on a single-core CPU.) By default, `Stan` uses a Hamiltonian Monte Carlo (HMC), No-U-Turns (NUTS) algorithm, automatically adapting step size and the mass matrix during warm-up, and drawing initial parameters (if unconstrained, as in our case) from a uniform distribution between -2 and 2. `Stan`'s diagnostics[4] (Tables 1, 2, 3) show that:

- the chains have converged, as Vehtari-Gelman's $\hat{R}$ statistic[5] is effectively 1;
- they mixed well, as the effective sample size (ESS) both in the bulk and in the tails of the distribution is comparable to the total number of posterior draws, 8000;
- the uncertainty on the parameters (standard deviation, SD, and mean absolute deviation, MAD) is also acceptable. In particular, the final SD is much smaller than that chosen for the original prior on each parameter.

Furthermore, the output of `cmdstan_diagnose()` states that:

- no divergent transitions were found in the Hamiltonian trajectory simulation steps;
- satisfactory "tree depth" for the NUTS algorithm was reported;

[3] Stan Development Team. *Stan Modeling Language Users Guide and Reference Manual*. `https://mc-stan.org`, 2026.

[4] Note that, in this section, parameters are expressed on the standardized scale. $\dot{T}$, in particular, expresses temperature increase per ten years, except in Oderzo station, where it has to be rescaled by $\frac{10}{9}$.

[5] Vehtari et al. (2021). Rank-normalization, folding, and localization: An improved $\hat{R}$ for assessing convergence of MCMC. *Bayesian Analysis*, 16(2)

- satisfactory Energy Bayesian Fraction of Missing Information (E-BFMI) was reported.

Note that $\hat{R}$ and ESS are calculated on the "rank normalized", "split" chains. Additionally, the autocorrelation plot (Figure 6) shows that the autocorrelation function (ACF) decays quickly for both parameters, indicating again that the chains have mixed well. Trace plots (Figure 5) also confirm what we have stated so far. Finally, we point out that the parameter correlations (Table 4) are generally small. This is also evident from the 2D scatter plots of the draws (Figure 8), where the samples appear broadly spread out when examining any pair of parameters[6].

Table 1: *Monte Summano - linear model's draws summary*

Parameter	mean	median	SD	MAD	5% quant.	95% quant.	$\hat{R}$	ESS_{bulk}	ESS_{tail}
T_0	11.4	11.4	0.11	0.11	11.2	11.6	1.0004	6240	5499
$\dot{T}$	0.7	0.7	0.12	0.12	0.5	0.9	1.001	7328	5393
σ	0.60	0.59	0.08	0.08	0.48	0.75	1.003	7219	5738

Table 2: *Oderzo - linear model's draws summary*

Parameter	mean	median	SD	MAD	5% quant.	95% quant.	$\hat{R}$	ESS_{bulk}	ESS_{tail}
T_0	13.3	13.3	0.09	0.08	13.1	13.4	1.0008	6785	5277
$\dot{T}$	0.52	0.52	0.09	0.09	0.37	0.68	1.0003	7286	5700
σ	0.48	0.47	0.07	0.06	0.38	0.60	1.0005	7180	5555

Table 3: *Cittadella - linear model's draws summary*

Parameter	mean	median	SD	MAD	5% quant.	95% quant.	$\hat{R}$	ESS_{bulk}	ESS_{tail}
T_0	13.71	13.71	0.09	0.09	13.57	13.86	1.0006	7346	5812
$\dot{T}$	0.6	0.6	0.10	0.10	0.4	0.8	1.001	6713	5354
σ	0.49	0.49	0.07	0.06	0.39	0.62	1.0003	6854	5580

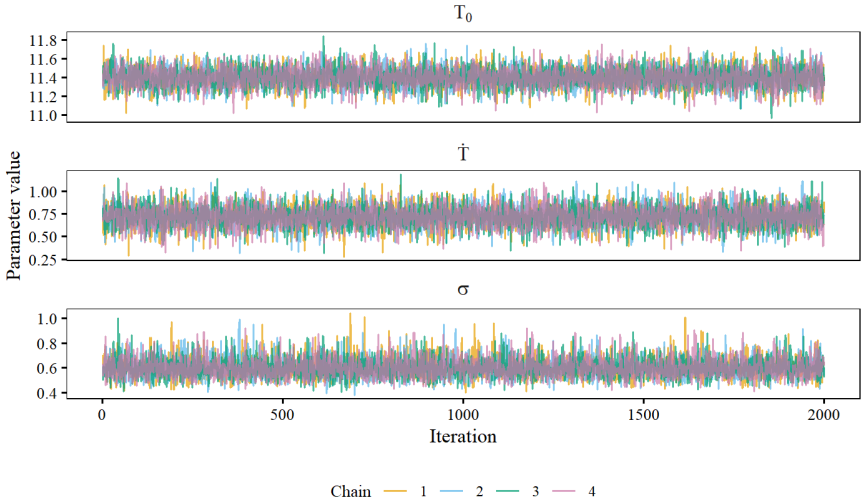
Table 4: *Parameter correlations for the linear model*

Correlation (%)	Monte Summano	Oderzo	Cittadella
T_0 vs. $\dot{T}$	-1.34	-0.91	-0.54
T_0 vs. σ	-1.47	-0.005	+0.43
$\dot{T}$ vs. σ	+1.16	+0.16	+0.38

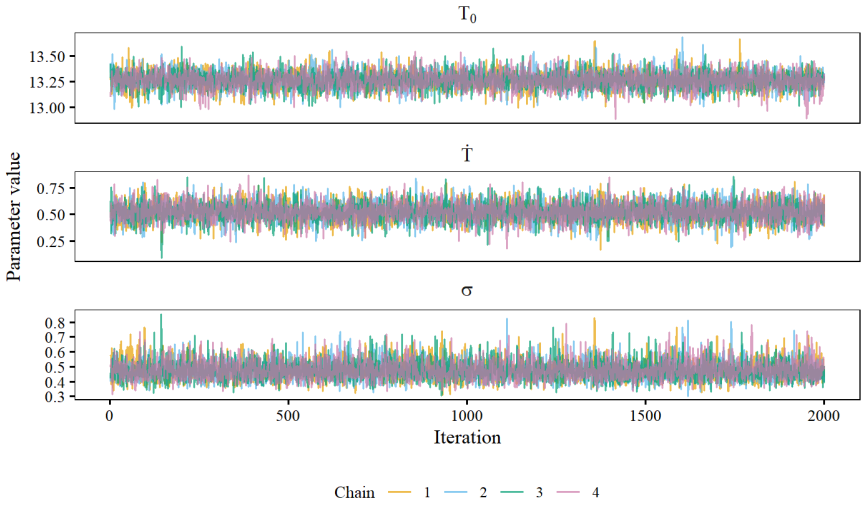
[6]A minor note is the following: in the correlation plots involving the σ_s parameter, the points seem more concentrated in the lower part (*i.e* near $\sigma_s = 0$). This can be explained by the fact that this parameter is drawn from a normal distribution centered in 0, but is restricted to positive values.

Figure 5: *Trace plots of all MCMC chains under the linear model, shown separately for each station.*

(a) *Monte Summano*



(b) *Oderzo*



(c) *Cittadella*

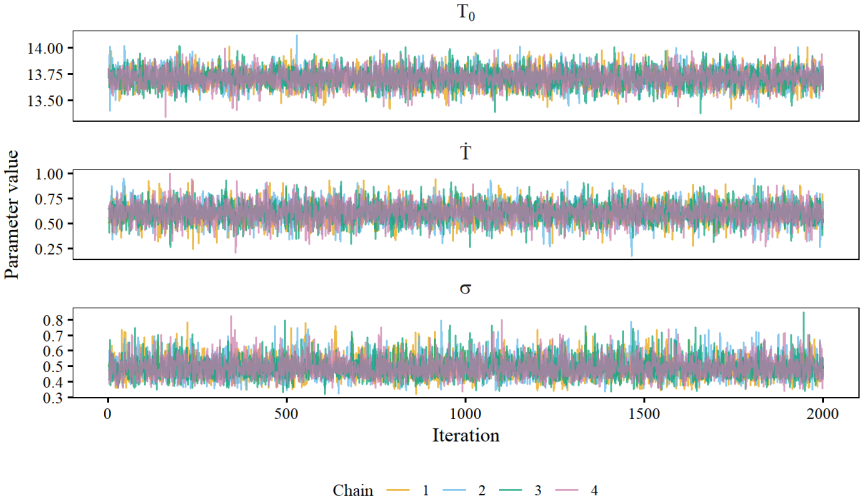
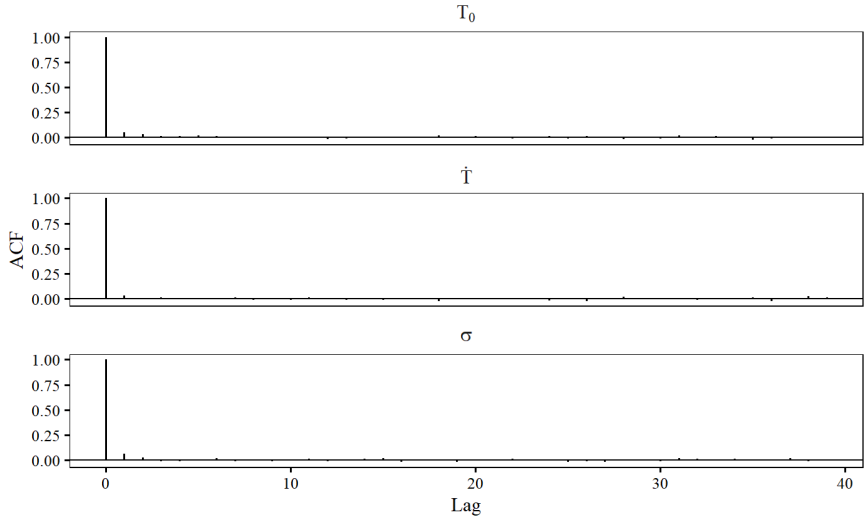
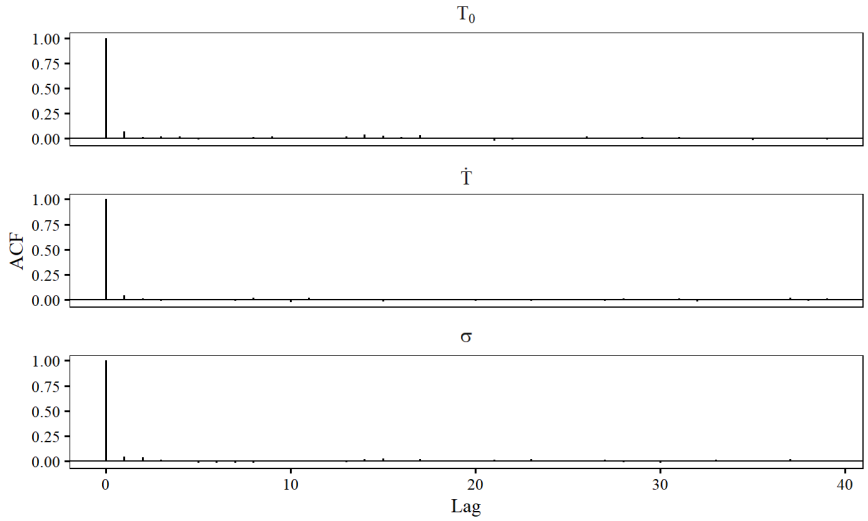


Figure 6: *Autocorrelation functions for MCMC chains under the linear model, shown separately for each station.*

(a) *Monte Summano*



(b) *Oderzo*



(c) *Cittadella*

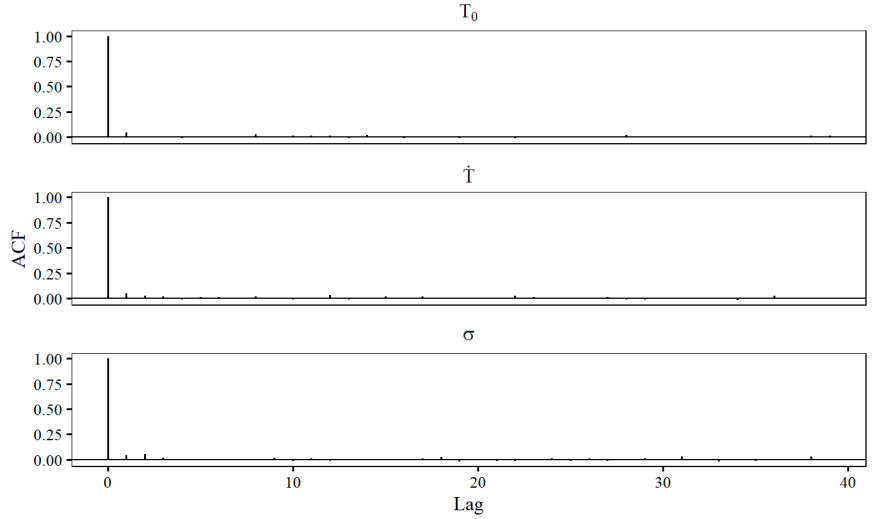
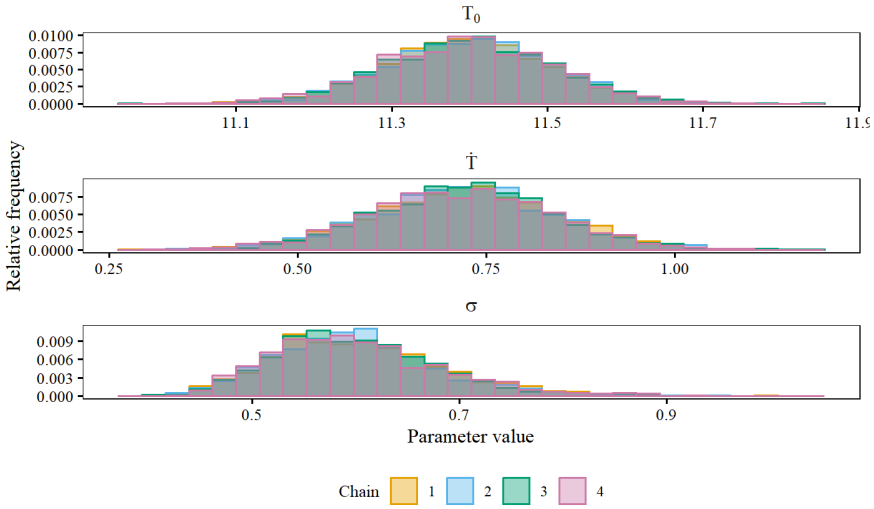
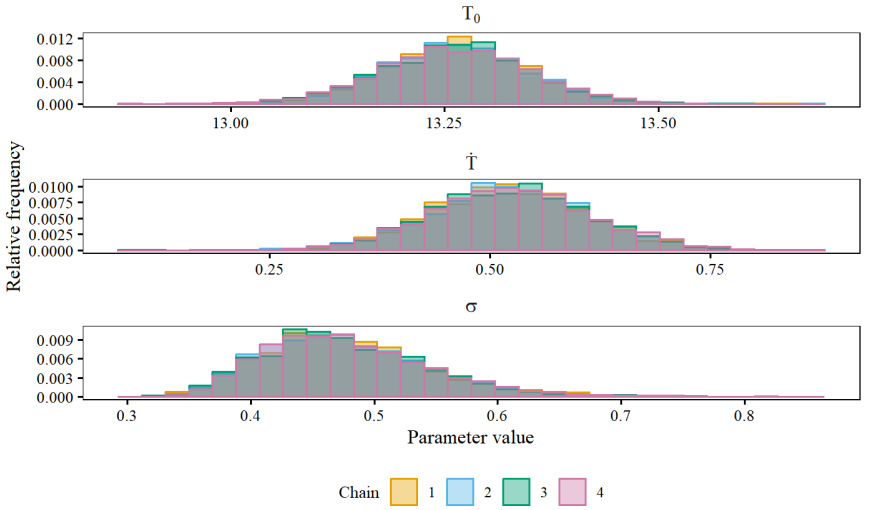


Figure 7: *Posterior marginal distributions from MCMC samples under the linear model, displayed separately for each station.*

(a) *Monte Summano*



(b) *Oderzo*



(c) *Cittadella*

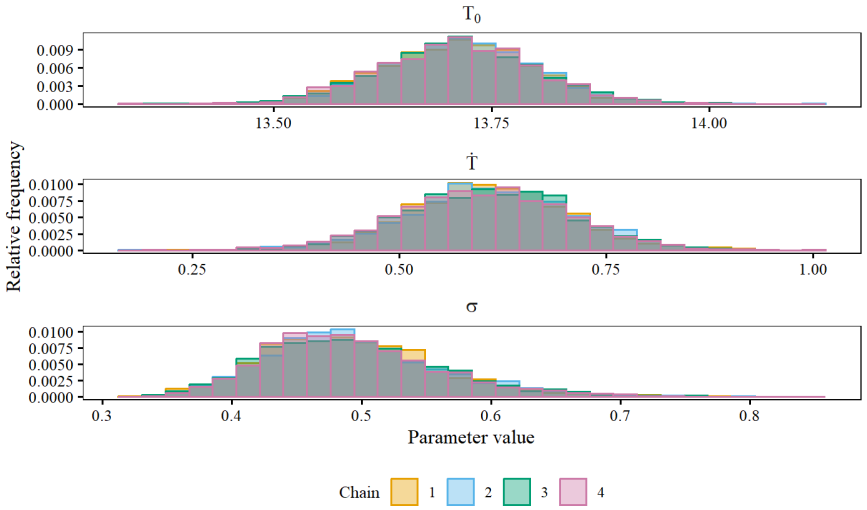
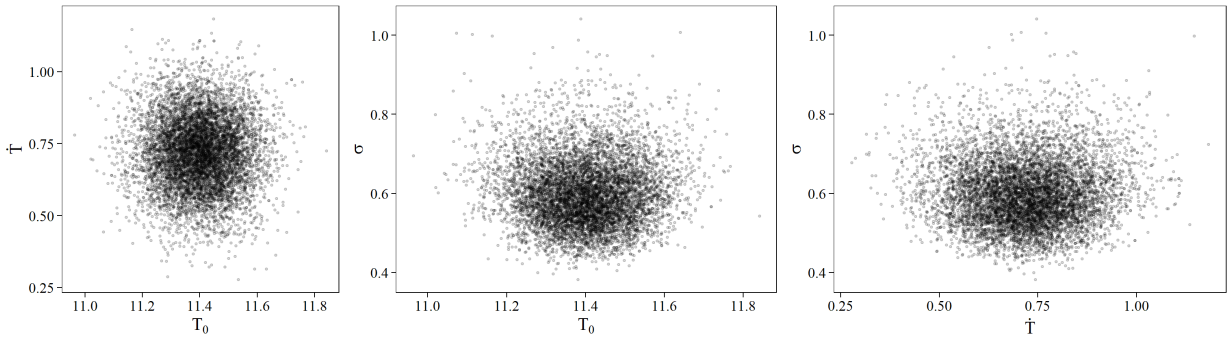
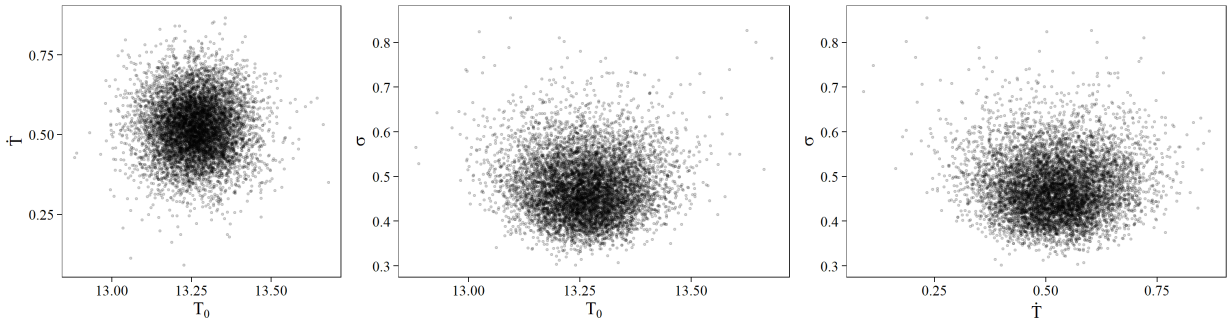


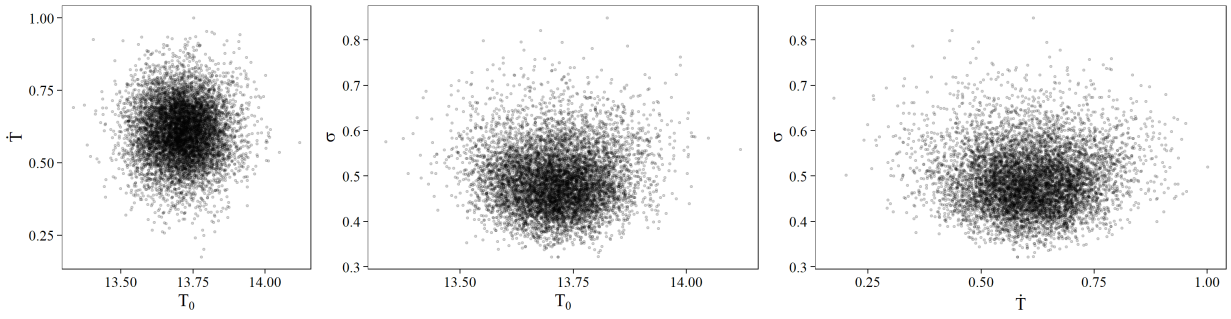
Figure 8: *Draws from the linear model fit, plotted in parameter space to illustrate correlations, for each station.*



(a) *Monte Summano*



(b) *Oderzo*



(c) *Cittadella*

3 Model Comparison

Constant-value model

We then perform the exact same fit procedure for each station for the constant-temperature model M_1. The corresponding `Stan` code (everything stays the same, except no $\dot{T}$ variable is present) is the following:

```
data {
int<lower=1> N;
vector[N] T;
vector[N] time; // unused
}

parameters {
real T0;
real<lower=0> sigma;
}

model {
T0 ~ normal(0, 20);
sigma ~ normal(0,10);

T ~ normal(T0, sigma);
}

generated quantities {
vector[N] log_lik;
for (i in 1:N)
log_lik[i] = normal_lpdf(T[i] | T0, sigma);
}
```

Results are presented in Table 6, 7, and 8 (draws summaries). As in the previous case, we also provide trace plots (Figure 9), ACFs (Figure 10), marginal density plots (Figure 11), and 2D draws scatter plots (Figure 12).

Table 5: *Parameter correlations for the constant model*

Correlation (%)	Monte Summano	Oderzo	Cittadella
T_0 vs. σ	0.29	1.50	-0.06

Table 6: *Monte Summano - constant model's draws summary*

Parameter	mean	median	SD	MAD	5% quant.	95% quant.	$\hat{R}$	ESS_{bulk}	ESS_{tail}
T_0	11.4	11.4	0.16	0.16	11.1	11.7	1.0008	5772	4888
σ	0.9	0.9	0.12	0.12	0.7	1.1	1.001	6791	5244

Table 7: *Oderzo - constant model's draws summary*

Parameter	mean	median	SD	MAD	5% quant.	95% quant.	$\hat{R}$	ESS_{bulk}	ESS_{tail}
T_0	13.3	13.3	0.13	0.13	13.1	13.5	1.0008	5580	4567
σ	0.7	0.7	0.10	0.09	0.6	0.9	1.0008	6036	5467

Table 8: *Cittadella - constant model's draws summary*

Parameter	mean	median	SD	MAD	5% quant.	95% quant.	$\hat{R}$	ESS_{bulk}	ESS_{tail}
T_0	13.7	13.7	0.14	0.14	13.5	13.9	1.001	4878	3980
σ	0.8	0.7	0.10	0.09	0.6	0.9	1.0005	5045	4398

As before, we do not encounter any particular issue during the fitting procedure.

We then perform model comparison between the linear-trend and constant-trend models using two complementary approaches: the Bayes factor and Leave-One-Out Cross-Validation (LOO-CV). These methods and corresponding results are presented in the following sections.

Bayes Factor Comparison

The Bayes factor is defined as the ratio of the marginal likelihoods (*i.e.*, the evidences) of two competing models. In this analysis, we consider BF_{21}, defined as the ratio between the evidence of the linear model (M_2) and that of the constant model (M_1). Assuming no prior preference between the two models, the Bayes factor coincides with the posterior odds. Model comparison therefore reduces to assessing whether $\frac{P(M_2|D)}{P(M_1|D)} > 1$, in which case there is evidence in favour of the linear model given the data. Conversely, values < 1 indicate support for the constant model. In particular, the Bayes factor is computed using the `bridgesampling` R package, which estimates the marginal likelihood via Monte Carlo integration. The method constructs a multivariate proposal distribution fitted to posterior samples and uses it within a bridge-sampling[7] framework to obtain stable estimates of model evidence. The results for the Monte Summano, Oderzo, and Cittadella stations are reported in Table 9. Across all three stations, the Bayes factor is substantially greater than

Table 9: *Bayes factors for model comparison across stations.*

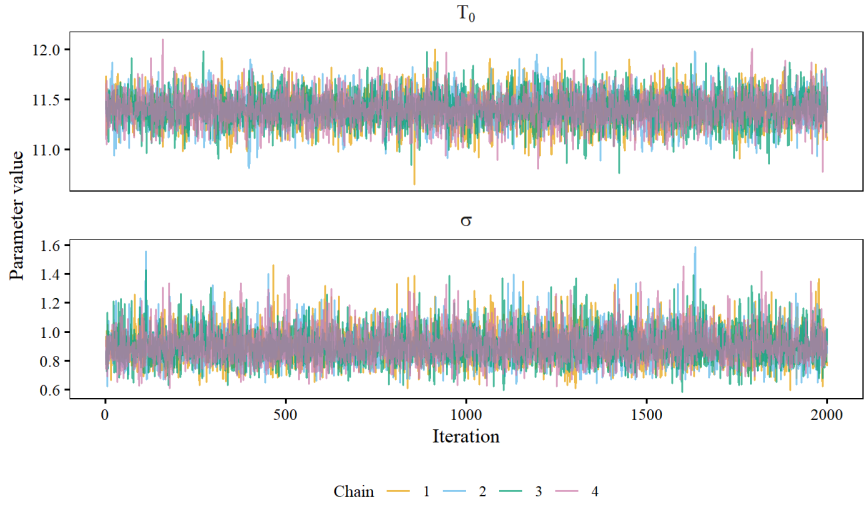
Station	**Bayes Factor (M_2 vs. M_1)**
Monte Summano	1306.9
Oderzo	549.6
Cittadella	1775.9

one, indicating strong evidence that the ARPAV measurements support a linear trend in mean annual temperature over time rather than a constant model. Although the Bayes factor for Oderzo is lower than for the other two stations, a value on the order of 10^2 still provides decisive evidence in favour of the linear trend model. As reported in Tables 1, 2, 3, the estimated slopes are positive for all stations. This suggests an increase in mean annual temperature over time according to the fitted model, consistent with a warming trend. Interestingly, Monte Summano presents a slightly more pronounced temperature raise; the reason, however, is unclear to us (the station is situated in the Prealps, but at only about 600 m a.s.l., compared to the other two which are located in the Veneto plain, at about sea level).

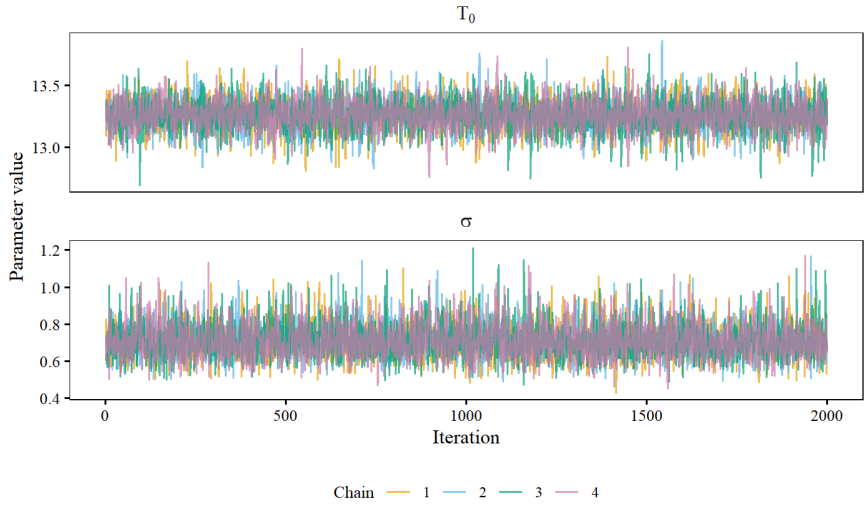
[7] Q. F. Gronau et al., "A Tutorial on Bridge Sampling," arXiv:1703.05984 (2017).

Figure 9: *Trace plots of all MCMC chains under the constant model, shown separately for each station.*

(a) *Monte Summano*



(b) *Oderzo*



(c) *Cittadella*

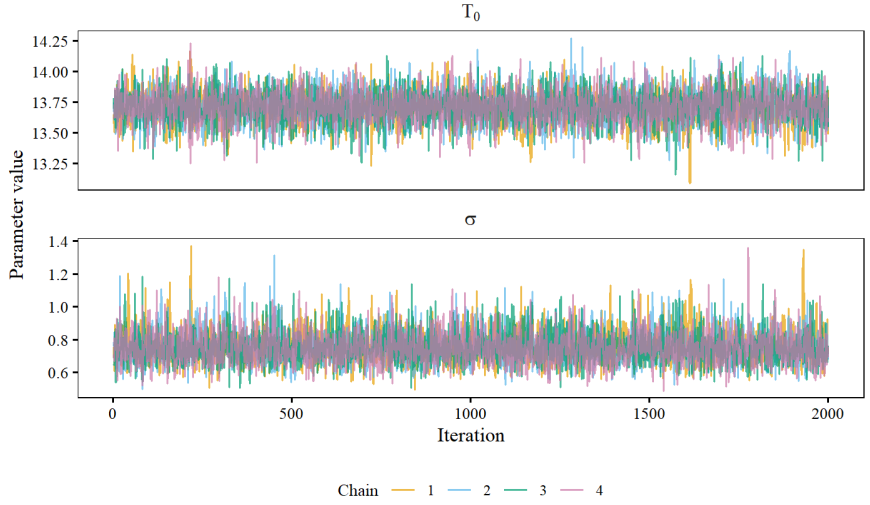
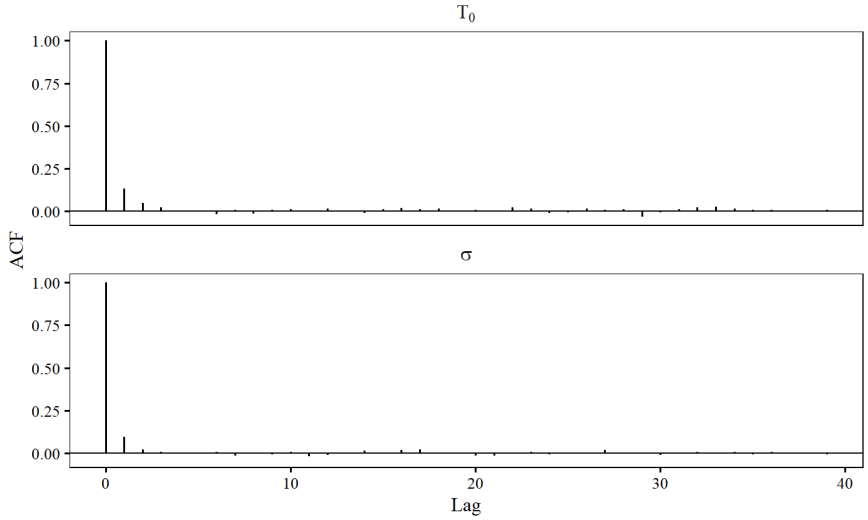
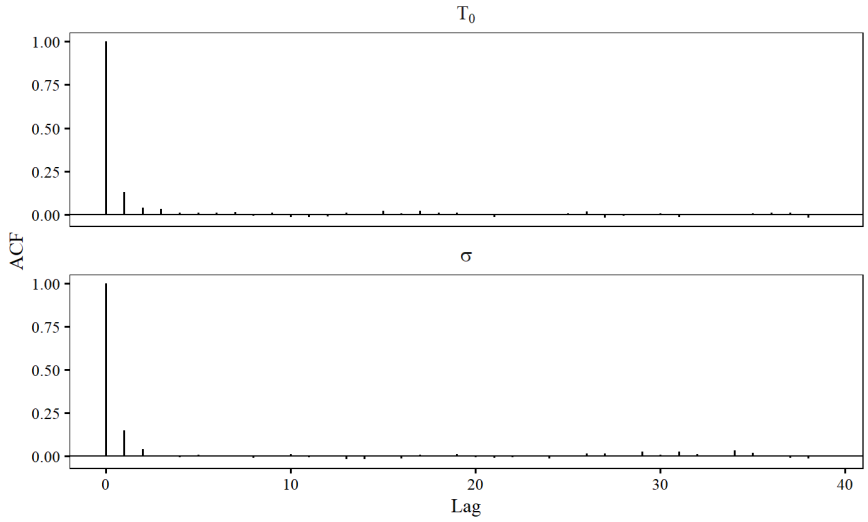


Figure 10: *Autocorrelation functions for MCMC chains under the constant model, shown separately for each station.*

(a) *Monte Summano*



(b) *Oderzo*



(c) *Cittadella*

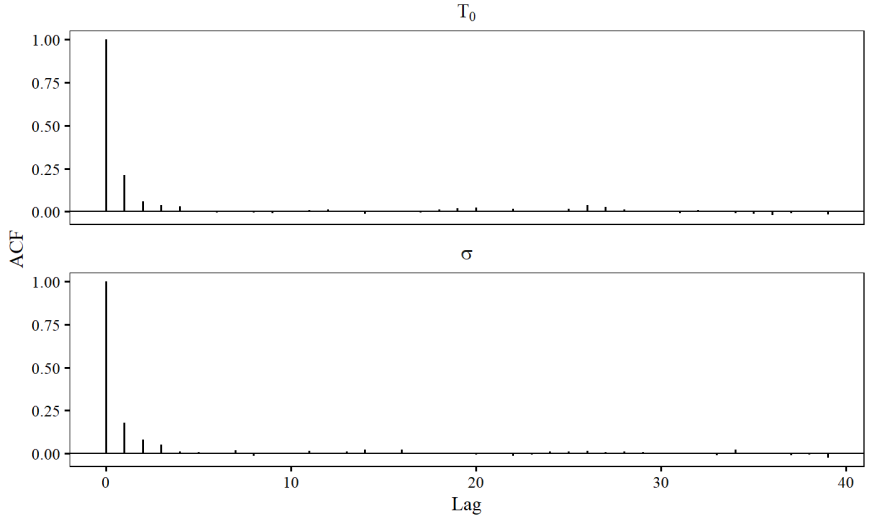
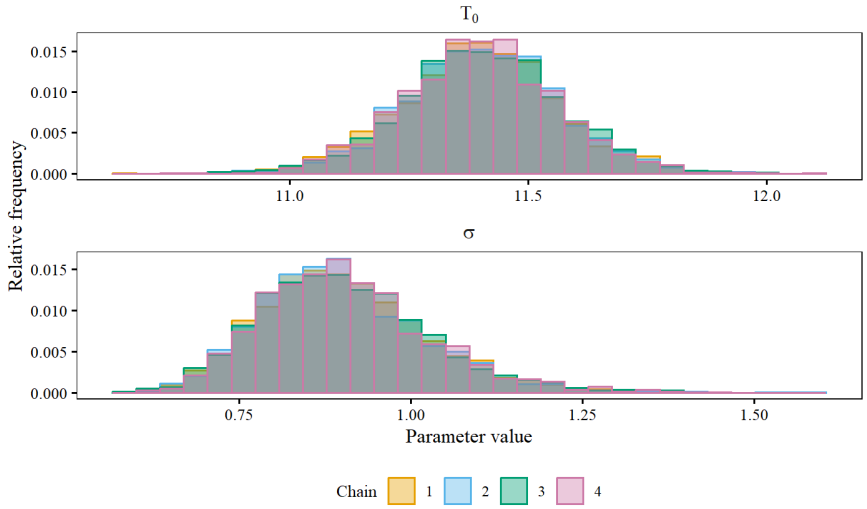
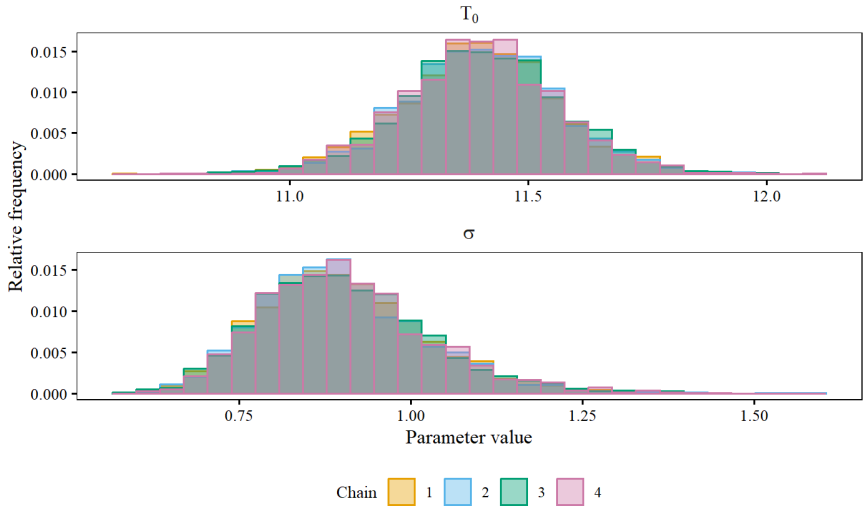


Figure 11: *Posterior marginal distributions from MCMC samples under the constant model, displayed separately for each station.*

(a) *Monte Summano*



(b) *Oderzo*



(c) *Cittadella*

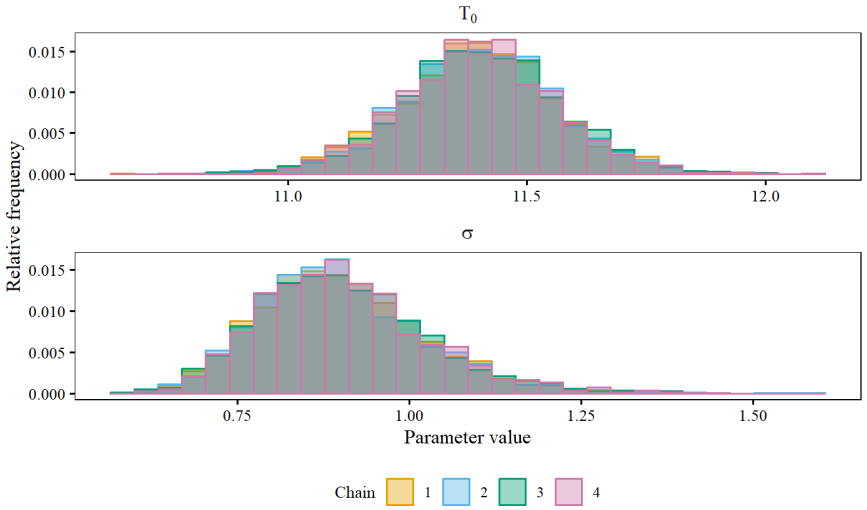
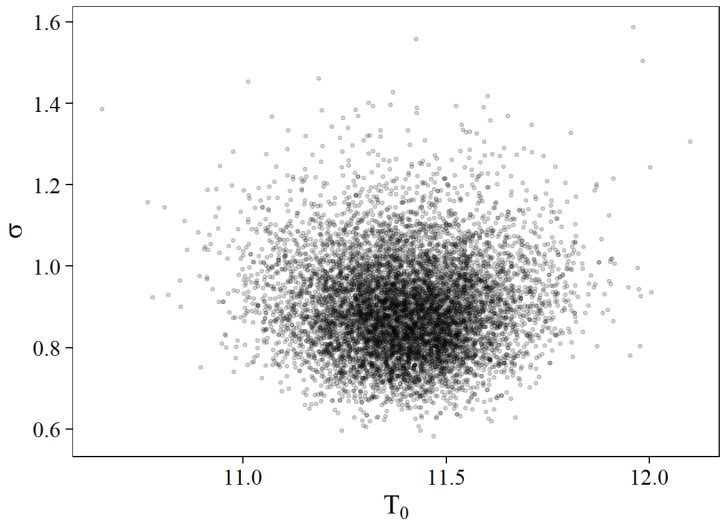
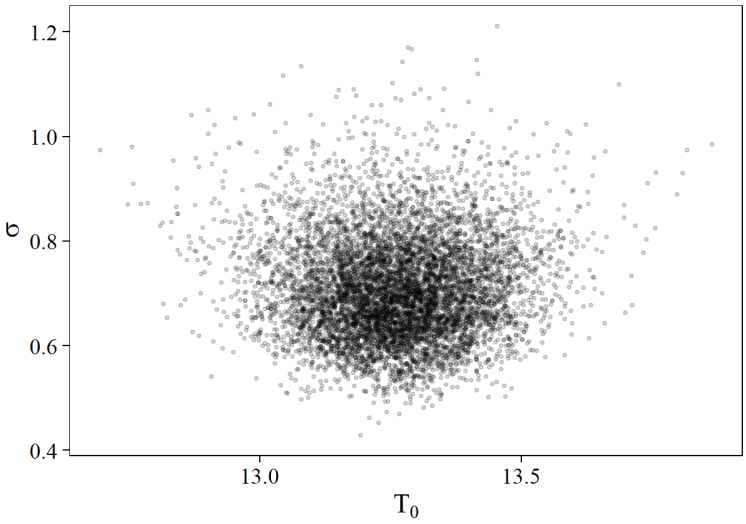


Figure 12: *Draws from the constant model fit, plotted in parameter space to illustrate correlations, for each station.*

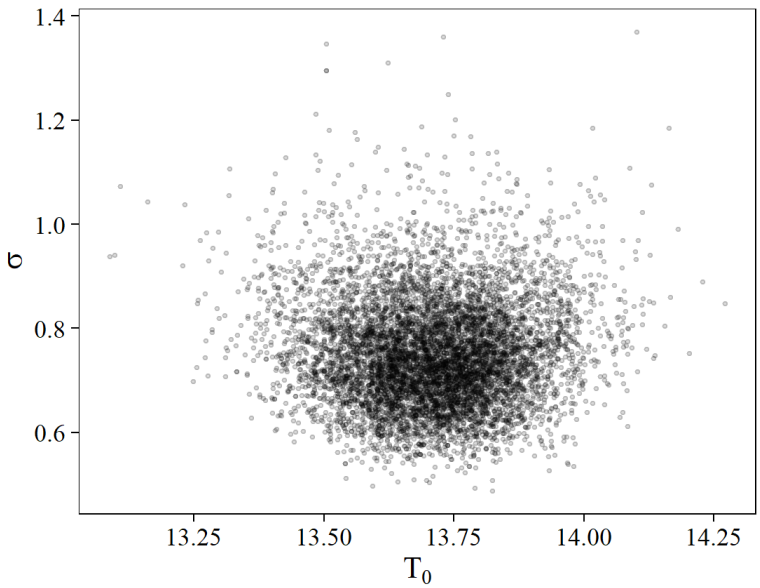
(a) *Monte Summano*



(b) *Oderzo*



(c) *Cittadella*



LOO-CV

Leave-One-Out Cross-Validation (LOOCV)[8] is a model comparison criterion based on the estimation of the expected log pointwise predictive density (ELPD), which quantifies a model's out-of-sample predictive accuracy for new observations. In LOOCV, the ELPD is computed as the sum of log predictive densities, where each observation is evaluated using a model fitted on the dataset excluding that observation. This procedure provides an estimate of predictive performance for unseen data drawn from the same data-generating process. In this context, each left-out observation can be interpreted as a proxy for an unobserved future measurement at the corresponding time point. In computational practice, as training N models is obviously unfeasible, total ELPD is approximated by means of a weighted sum on predictive probabilities, using importance weights derived from the full-data posterior (from the one fitted model). A further issue is weight instability in the tails of the distribution (large weights): for this reason, tail weights are fitted to a Pareto distribution (this is called Pareto smoothed importance sampling or PSIS), whose estimated parameter $\hat{k}$ is used to determine whether the metric is sound for a given problem. (In our case, upon checking diagnostics (in particular, a Pareto $\hat{k} < 0.7$ for both models), we deem the metric suitable.) Therefore, a higher ELPD indicates better expected predictive performance. Under the assumption that the underlying climatic dynamics remain stable over time, a model with higher ELPD is expected to yield more accurate predictions of future temperature values. In this analysis, for each station the ELPD difference between the two models (linear minus constant) and its associated standard error were computed using the `loo` R package. The computation is based on pointwise expected log predictive densities, which are summed over observations to obtain the total ELPD difference, while the associated uncertainty is derived from the variability of the pointwise differences. The results are reported in Table 10.

Table 10: *LOO-CV ELPD differences between linear and constant models across locations.*

Location	ΔELPD (Lin − Const)	**Std. Error**
Monte Summano	11.2	5.9
Oderzo	11.4	3.1
Cittadella	12.6	3.6

Across all stations, the ELPD difference is positive and its magnitude is approximately three times the associated standard error, indicating strong evidence that the linear model outperforms the constant model in out-of-sample predictive accuracy under the stated assumptions.

4 Posterior visualization with predictive inference

For each model and station, we visualize data points, the fit line, and its uncertainty. We recall that, for each time step t (the year), the predicted temperature is given by the model's response function $T_{s,t} = T_s^0 + \dot{T}_s\, t$ averaged over *all* $(T_s^0, \dot{T}_s)$ MC chains samples (obviously $\dot{T}_s = 0$ for M_1). The uncertainty on the fit line is instead computed (for each t) as a 95% centered CI, *i.e.* one shows *the 2.5% and 97.5% quantiles* of the model's response function (the same as before) calculated for each chain sample. For each model, we also compute a ten-year temperature prediction and its uncertainty. Calculating the prediction line itself is straightforward, as it is simply an extension of the resulting response function to years for which no data is available yet. The uncertainty on this prediction (*i.e.* on new observations) is perhaps less intuitive but equally simple to calculate: since future measurements will include noise (whose level, we assume, will not change over time), the function to calculate over chain samples is $T_{s,t} = T_s^0 + \dot{T}_s\, t + \epsilon_{s,t}, \quad \epsilon_{s,t} \sim \mathcal{N}(0, \sigma_s^2)$. In other words, measurement noise is included in the sense that for each chain sample $(T_s^0, \dot{T}_s, \sigma_s)$ we draw *a single* sample from $\mathcal{N}(T_s^0 + \dot{T}_s\, t, \quad \sigma_s)$; then, as before, we take the 2.5% and 97.5% quantiles of this quantity. All this is shown in in Figure 13. For clarity, all uncertainties are computed directly in R code, without resorting to Stan's automation.

[8] Vehtari, Gelman & Gabry (2017). Practical Bayesian model evaluation using leave-one-out cross-validation and WAIC. *Statistics and Computing*, 27:1413–1432.

In Figures 14, 15, and 16, we show the full predictive posterior distribution of temperature for each of the 10 forecast years, for both the constant (red) and linear (black) models, respectively for Monte Summano, Oderzo, and Cittadella. We note in particular how the average temperature increase per decade, $\dot{T}$, is about 0.7 degrees Celsius (also see Tables 1, 3, 2).

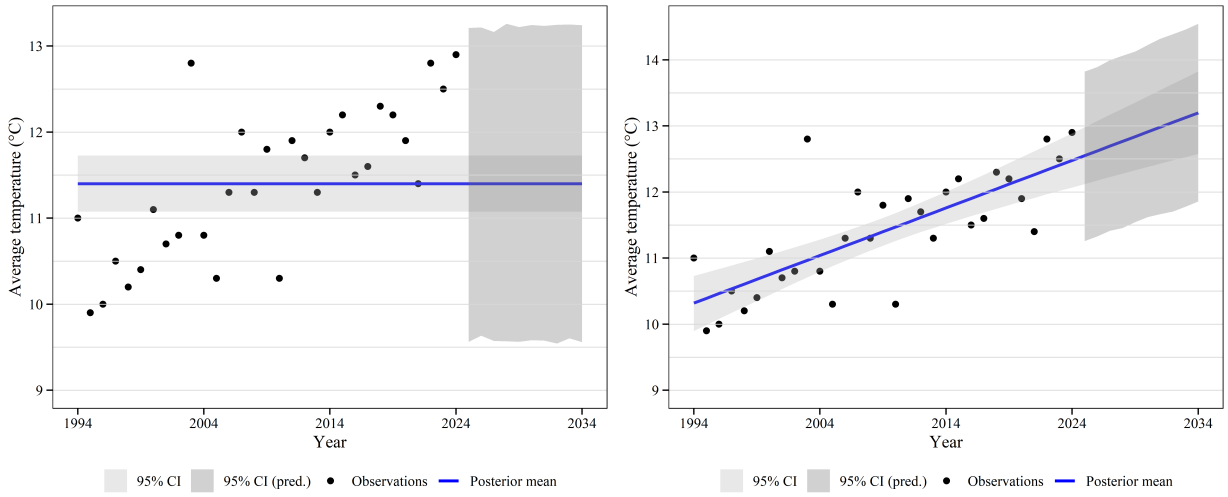
Visually, we have the impression that the linear (increasing temperature) model is a better fit to the data. In fact, the constant model assigns higher noise levels to all stations (compare σ_s values in the respective tables), seemingly to compensate for its predictive deficiencies. This confirms qualitatively what quantitative model comparison already told us.

5 Conclusion

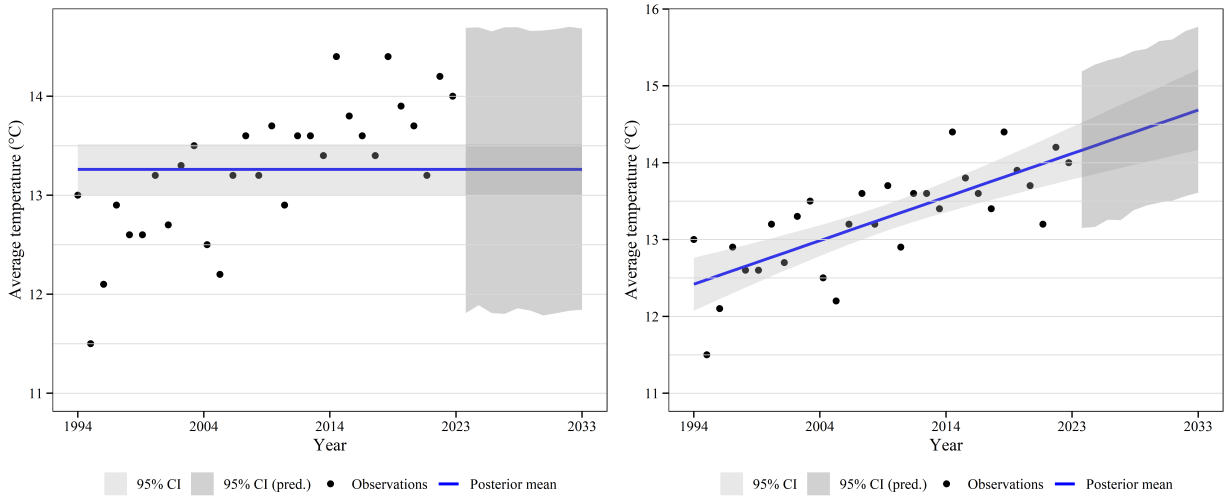
We analysed 30 years of ARPAV temperature data from three Veneto stations using Bayesian linear and constant models. Both Bayes factors and LOO-CV strongly support a positive linear trend, with estimated warming rates of 0.6-0.7 °C per decade. The Monte Summano station shows the steepest rise, possibly due to regional elevation-dependent effects, though local factors cannot be excluded. Posterior predictive distributions for the next decade indicate continued warming under the linear model, while the constant model yields wider uncertainty. Overall, the evidence for systematic upward temperature trends in the Veneto region is robust, although longer time series and additional stations would strengthen inference.

Figure 13: *Comparison of constant (left) and linear (right) fits, showing the mean regression curve with credible intervals and the predictive credible band.*

(a) *Monte Summano*



(b) *Oderzo*



(c) *Cittadella*

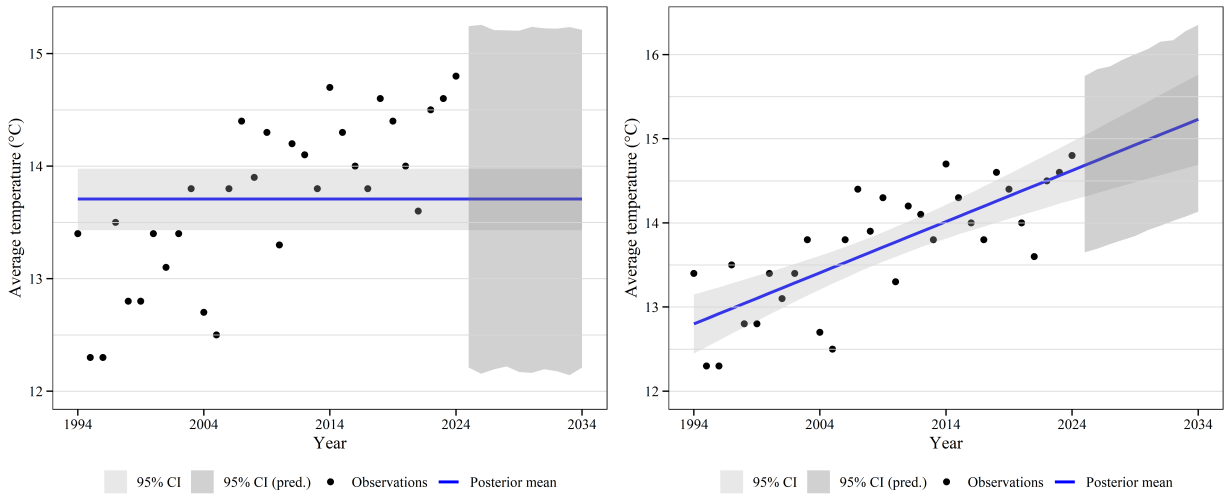


Figure 14: *Posterior predictive distributions for the next 10 years in Monte Summano: comparison of linear and constant models*

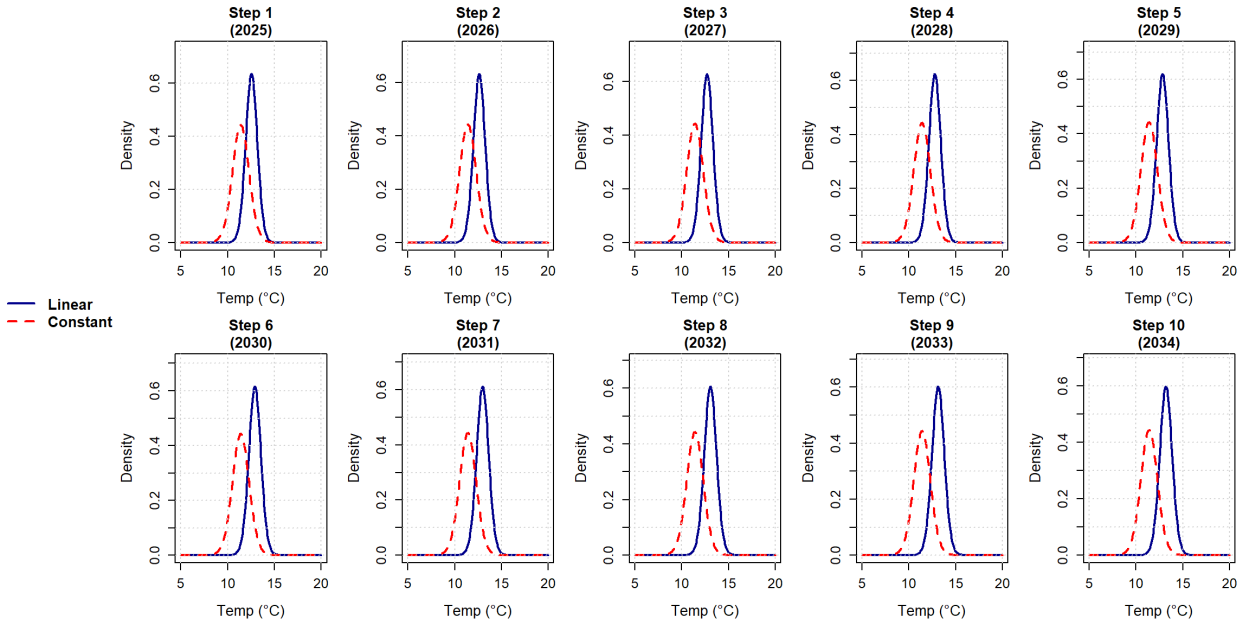


Figure 15: *Posterior predictive distributions for the next 10 years in Oderzo: comparison of linear and constant models*

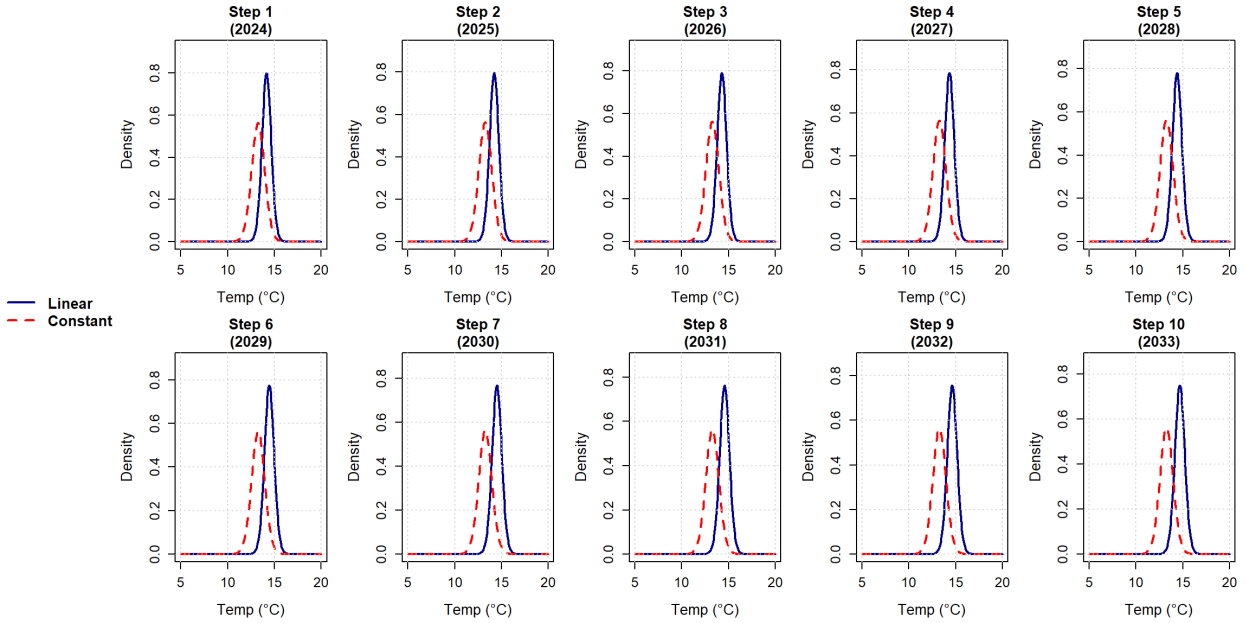
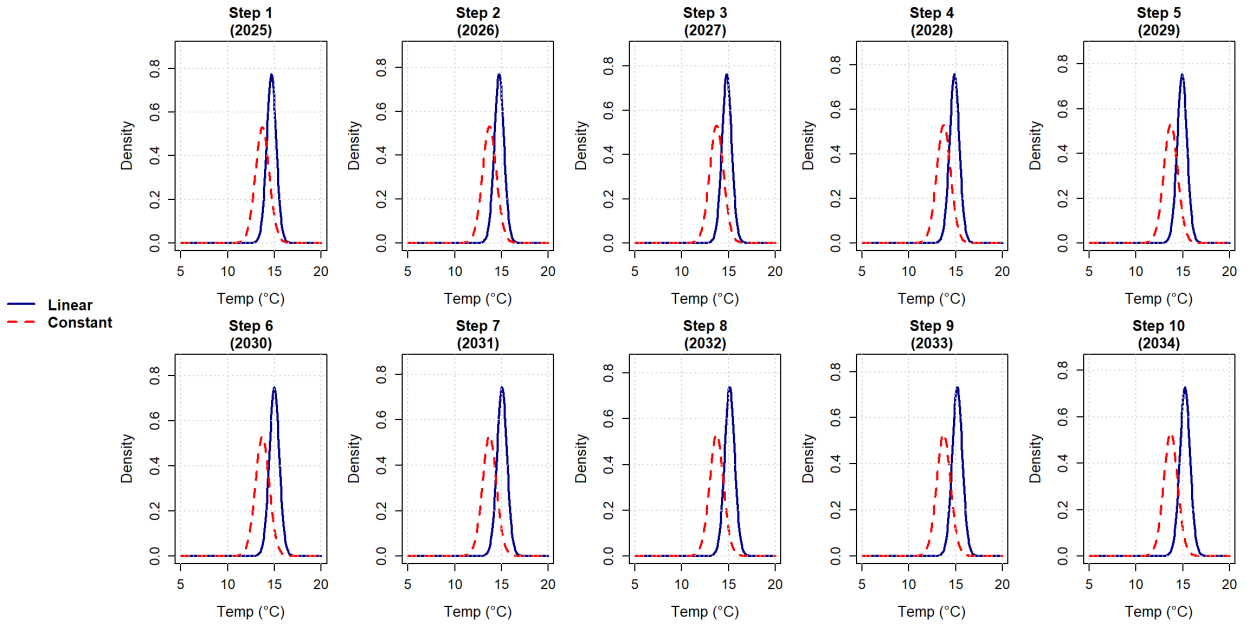


Figure 16: *Posterior predictive distributions for the next 10 years in Cittadella: comparison of linear and constant models*



R code

The complete R code (for all sections) follows, commented where necessary. (`.stan` files code is not included here.)

Function definitions (`functions.R`)

```
# PACKAGES

library(readr)
library(cmdstanr)
library(posterior)
library(ggplot2)
library(loo)
library(bridgesampling)
library(patchwork)

# DATA READING FUNCTION

find_data <- function(path){
  row <- read_lines(path, locale = locale(encoding = "latin1", decimal_mark = "."))

  row_min <- grep("Temperatura", row)[1]
  index_1994_min <- row_min + grep("^1994;", row[row_min:length(row)])[1] - 1
  index_2024_min <- row_min + grep("^2024;", row[row_min:length(row)])[1] - 1

  row_mean <- grep("Temperatura", row)[2]
  index_1994_mean <- row_mean + grep("^1994;", row[row_mean:length(row)])[1] - 1
  index_2024_mean <- row_mean + grep("^2024;", row[row_mean:length(row)])[1] - 1

  row_max <- grep("Temperatura", row)[3]
  index_1994_max <- row_max + grep("^1994;", row[row_max:length(row)])[1] - 1
  index_2024_max <- row_max + grep("^2024;", row[row_max:length(row)])[1] - 1
  n_row <- index_2024_max - index_1994_max + 1
```

```
  data_min <- read_delim(
    path,
    delim = ";",
    col_names = c("Anno", "GEN", "FEB", "MAR", "APR", "MAG", "GIU",
                  "LUG", "AGO", "SET", "OTT", "NOV", "DIC", "Medio"),
    skip = index_1994_min - 1,
    n_max = n_row,
    col_types = cols(.default = col_double()),
    locale = locale(encoding = "latin1", decimal_mark = "."),
    na = c("", "NA", ">>")
  )

  data_mean <- read_delim(
    path,
    delim = ";",
    col_names = c("Anno", "GEN", "FEB", "MAR", "APR", "MAG", "GIU",
                  "LUG", "AGO", "SET", "OTT", "NOV", "DIC", "Medio"),
    skip = index_1994_mean - 1,
    n_max = n_row,
    col_types = cols(.default = col_double()),
    locale = locale(encoding = "latin1", decimal_mark = "."),
    na = c("", "NA", ">>")
  )

  data_max <- read_delim(
    path,
    delim = ";",
    col_names = c("Anno", "GEN", "FEB", "MAR", "APR", "MAG", "GIU",
                  "LUG", "AGO", "SET", "OTT", "NOV", "DIC", "Medio"),
    skip = index_1994_max - 1,
    n_max = n_row,
    col_types = cols(.default = col_double()),
    locale = locale(encoding = "latin1", decimal_mark = "."),
    na = c("", "NA", ">>")
  )

  return(list(data_min, data_mean, data_max))
}

# DATA PLOTTING FUNCTION

plot_hist <- function(temp_min, temp_mean, temp_max, month = "", city = "") {
  year <- if (city == "Oderzo") 1994:2023 else 1994:2024
  t_min <- min(c(temp_min, temp_mean, temp_max), na.rm = TRUE)
  t_max <- max(c(temp_min, temp_mean, temp_max), na.rm = TRUE)

  old_par <- par(no.readonly = TRUE)
  on.exit(par(old_par))
  par(mar = c(5, 4, 4, 7) + 0.1)

  if (month == "Mean") {
    title <- paste("Average Temperature -", city)
  } else {
    title <- paste("Temperature in", month, "-", city)
  }

  plot(year, temp_max,
       type = "n",
       ylim = c(-5, 35),
       main = title,
       xlab = "Year",
       ylab = "Temperature (°C)")

  grid(nx = NULL,
       ny = NULL,
       col = "lightgray",
       lty = "dotted",
       lwd = 1)

  lines(year, temp_max,
```

```
        type = "h",
        lwd = 10,
        lend = 1,
        pch = 16,
        col = "red", border = "white")

  lines(year, temp_mean,
        type = "h",
        lwd = 10,
        lend = 1,
        pch = 16,
        col = "darkorange", border = "white")

  lines(year, temp_min,
        type = "h",
        lwd = 10,
        lend = 1,
        pch = 16,
        col = "blue", border = "white")

  abline(h = 0, col = "gray50", lwd = 1, lty = 2)

  legend("topright",
         legend = c("Max", "Mean", "Min"),
         fill = c("red", "darkorange", "blue"),
         border = NA,
         bty = "n",
         xpd = TRUE,
         inset = c(-0.22, 0))
}

# SUBPLOT PLOTTING

plot_hist_all <- function(data, city = "") {

  months <- c("GEN", "FEB", "MAR", "APR", "MAG", "GIU",
              "LUG", "AGO", "SET", "OTT", "NOV", "DIC")

  month_name <- c("January", "February", "March", "April", "May", "June",
                  "July", "August", "September", "October", "November", "December")

  x <- if (city == "Oderzo") 1994:2023 else 1994:2024

  all_min  <- unlist(lapply(data[[1]][months], identity))
  all_mean <- unlist(lapply(data[[2]][months], identity))
  all_max  <- unlist(lapply(data[[3]][months], identity))

  y_lim <- range(c(all_min, all_mean, all_max), na.rm = TRUE)

  old_par <- par(no.readonly = TRUE)
  on.exit(par(old_par))

  par(mfrow = c(4, 3),
      mar = c(4, 4, 3, 1),
      oma = c(5, 5, 5, 1),

      cex.axis = 1.4,
      font.axis = 2,

      cex.lab = 1.5,
      font.lab = 2,

      mgp = c(3, 1.2, 0),
      tck = 0.03)

  for (i in seq_along(months)) {

    m <- months[i]

    temp_min  <- data[[1]][[m]]
```

```
    temp_mean <- data[[2]][[m]]
    temp_max  <- data[[3]][[m]]

    plot(x, temp_max,
         type = "n",
         ylim = y_lim,
         xlim = range(x),
         main = month_name[i],
         cex.main = 1.9,
         font.main = 2,
         xlab = "",
         ylab = "")

    grid(col = "lightgray", lty = "dotted")

    lines(x, temp_max,  type = "h", lwd = 9, col = "red", lend = 1)
    lines(x, temp_mean, type = "h", lwd = 9, col = "darkorange", lend = 1)
    lines(x, temp_min,  type = "h", lwd = 9, col = "blue", lend = 1)

    abline(h = 0, col = "gray50", lty = 2)

    if (i == 1) {
      legend("topright",
             legend = c("Max", "Mean", "Min"),
             fill = c("red", "darkorange", "blue"),
             border = NA,
             bty = "n",
             cex = 1.5,
             text.font = 2,
             pt.cex = 2)
    }
  }

  mtext(paste("Min-Mean-Max Temperature -", city),
        outer = TRUE,
        line = 2,
        cex = 1.8,
        font = 2)

  mtext("Year",
        side = 1,
        outer = TRUE,
        line = 2.5,
        cex = 1.6,
        font = 2)

  mtext("Temperature (°C)",
        side = 2,
        outer = TRUE,
        line = 2.5,
        cex = 1.6,
        font = 2)
}

# STAN FIT FUNCTION

build_fit_stan_model <- function(stan_file_path, my_data) {
  my_mod <- cmdstan_model(stan_file_path)
  cat("MODEL COMPILED SUCCESSFULLY\n")
  my_fit <- my_mod$sample(
    data = my_data,
    chains = 4,
    parallel_chains = 4,
    iter_warmup = 1000,
    iter_sampling = 2000,
    seed = 222
  )
  return(list(mod = my_mod, fit = my_fit))
}
```

```r
# CORRELATIONS FUNCTION

plot_2_correlations <- function(my_draws_df, params_list, axes_names = c(expression(T[0]), expression(dot(T)))) {
  cm <- cor(as.matrix(my_draws_df[, params_list]))
  p <- ggplot(my_draws_df, aes(x = .data[[params_list[1]]], y = .data[[params_list[2]]])) +
    geom_point(alpha = 0.15, size = 0.8, color = "black") +
    coord_fixed(ratio = 1) +
    labs(
      x = axes_names[1],
      y = axes_names[2]
    ) +
    theme_classic(base_size = 14, base_family = "serif") +
    theme(
      panel.border = element_rect(fill = NA, color = "black", linewidth = 0.5),
      axis.line = element_blank(),
      axis.ticks = element_line(color = "black", linewidth = 0.5),
      axis.ticks.length = unit(0.15, "cm"),
      axis.title = element_text(size = 16),
      axis.text = element_text(size = 13, color = "black")
    )
  return(list(cor = cm, plot = p))
}

# POSTERIOR PLOT

plot_posterior <- function(my_draws_df, my_x_stdd, my_y_data,
                           my_x_grid = NULL, n_grid = N, n_pred_grid = N_fut_steps,
                           orig_mean = years_mean, orig_sd = years_sd_ceiling, my_step = step,
                           xb_l = 5, xl_l = 5, yg_b = 0.5) {
  if (is.null(my_x_grid)) {
    x_grid <- seq(min(my_x_stdd), max(my_x_stdd) + my_step * n_pred_grid, length.out = n_grid + n_pred_grid)
  } else {
    x_grid <- my_x_grid
  }

  S <- nrow(my_draws_df)
  T_mat <- matrix(NA, nrow = S, ncol = length(x_grid))

  if (!"T_dot" %in% colnames(my_draws_df)) {
    T_dot_vals <- rep(0, S)
  } else {
    T_dot_vals <- my_draws_df$T_dot
  }

  for (i in seq_along(x_grid)) {
    T_mat[, i] <- my_draws_df$T0 + T_dot_vals * x_grid[i]
  }

  T_mean <- apply(T_mat, 2, mean)
  T_lo <- apply(T_mat, 2, quantile, 0.025)
  T_hi <- apply(T_mat, 2, quantile, 0.975)
  T_pred_mat <- matrix(NA, S, length(x_grid))
  for (i in 1:S) {
    T_pred_mat[i, ] <- rnorm(length(x_grid), mean = T_mat[i, ], sd = my_draws_df$sigma[i])
  }
  T_pred_lo <- apply(T_pred_mat, 2, quantile, 0.025)
  T_pred_hi <- apply(T_pred_mat, 2, quantile, 0.975)

  plot_df <- data.frame(x_stdd = x_grid, mean = T_mean, lo = T_lo, hi = T_hi, p_lo = T_pred_lo, p_hi = T_pred_hi)

  p <- ggplot() +
    geom_point(aes(x = my_x_stdd, y = my_y_data, color = "Observations")) +
    geom_line(data = plot_df, aes(x = x_stdd, y = mean, color = "Posterior mean"), linewidth = 1) +
    geom_ribbon(data = plot_df, aes(x = x_stdd, ymin = lo, ymax = hi, fill = "95% CI"), alpha = 0.3) +
    geom_ribbon(data = plot_df[(n_grid + 1):(n_grid + n_pred_grid), ], aes(x = x_stdd, ymin = p_lo, ymax = p_hi, fill = "95% CI
    scale_color_manual(values = c("Observations" = "black", "Posterior mean" = "blue")) +
    scale_fill_manual(values = c("95% CI" = "grey70", "95% CI (pred.)" = "grey40")) +
    labs(x = "Year", y = "Average temperature (°C)") +
    guides(color = guide_legend(title = NULL), fill = guide_legend(title = NULL)) +
    scale_x_continuous(
```

```
      breaks = seq(min(x_grid), max(x_grid), length.out = xb_l),
      labels = round(orig_mean + orig_sd * seq(min(x_grid), max(x_grid), length.out = xl_l))
    ) +
    geom_hline(
      yintercept = seq(floor(min(my_y_data)), floor(max(T_pred_hi)), by = yg_b),
      color = "grey85", linewidth = 0.3
    ) +

    theme_classic(base_size = 12, base_family = "serif") +
    theme(
      panel.border = element_rect(fill = NA, color = "black", linewidth = 0.5),
      axis.line = element_blank(),
      axis.ticks = element_line(color = "black", linewidth = 0.5),
      axis.ticks.length = unit(0.15, "cm"),
      panel.grid.major = element_blank(),
      panel.grid.minor = element_blank(),
      axis.title = element_text(size = 12),
      axis.text = element_text(size = 10, color = "black"),
      legend.position = "bottom",
      legend.direction = "horizontal",
      legend.box = "horizontal",
      legend.title = element_blank(),
      legend.text = element_text(size = 10, family = "serif"),
      legend.margin = margin(t = 0, b = 0, unit = "pt"),
      legend.spacing.x = unit(5, "pt")
    )

  return(p)
}

# GRAPHS: Traces, ACFs, densities

plot_various_graphs <- function(my_fit, params_list) {
  draws_array <- as.array(my_fit$draws())
  n_iters <- dim(draws_array)[1]
  n_chains <- dim(draws_array)[2]

  df_list <- list()
  idx <- 1
  for (p in params_list) {
    for (chain in 1:n_chains) {
      vals <- draws_array[, chain, p]
      df_list[[idx]] <- data.frame(
        Parameter = p,
        Chain = factor(chain),
        Iteration = seq_along(vals),
        Value = as.numeric(vals),
        stringsAsFactors = FALSE
      )
      idx <- idx + 1
    }
  }
  df_long <- do.call(rbind, df_list)

  label_map <- c("T0" = "T[0]", "T_dot" = "dot(T)", "sigma" = "sigma")
  df_long$Parameter <- factor(df_long$Parameter,
                              levels = params_list,
                              labels = label_map[params_list]
  )

  trace_plot <- ggplot(df_long, aes(x = Iteration, y = Value, color = Chain)) +
    geom_line(alpha = 0.7) +
    facet_wrap(~Parameter, scales = "free_y", ncol = 1, labeller = label_parsed) +
    labs(x = "Iteration", y = "Parameter value") +
    scale_color_manual(values = c("1" = "#E69F00", "2" = "#56B4E9", "3" = "#009E73", "4" = "#CC79A7")) +
    theme_classic(base_size = 12, base_family = "serif") +
    theme(
      panel.border = element_rect(fill = NA, color = "black", linewidth = 0.5),
      axis.line = element_blank(),
      strip.background = element_blank(),
```

```
      strip.text = element_text(face = "bold", size = 12, family = "serif"),
      legend.position = "bottom",
      legend.title = element_text(size = 10),
      legend.text = element_text(size = 9)
    )

  hist_plot <- ggplot(df_long, aes(x = Value, fill = Chain, color = Chain)) +
    geom_histogram(aes(y = after_stat(count / sum(count))), alpha = 0.4, bins = 30, position = "identity") +
    facet_wrap(~Parameter, scales = "free", ncol = 1, labeller = label_parsed) +
    labs(x = "Parameter value", y = "Relative frequency") +
    scale_color_manual(values = c("1" = "#E69F00", "2" = "#56B4E9", "3" = "#009E73", "4" = "#CC79A7")) +
    scale_fill_manual(values = c("1" = "#E69F00", "2" = "#56B4E9", "3" = "#009E73", "4" = "#CC79A7")) +
    theme_classic(base_size = 12, base_family = "serif") +
    theme(
      panel.border = element_rect(fill = NA, color = "black", linewidth = 0.5),
      axis.line = element_blank(),
      strip.background = element_blank(),
      strip.text = element_text(face = "bold", size = 12, family = "serif"),
      legend.position = "bottom",
      legend.title = element_text(size = 10),
      legend.text = element_text(size = 9)
    )

  acf_list <- list()
  for (param in params_list) {
    param_draws <- as.vector(my_fit$draws(param))
    acf_obj <- acf(param_draws, plot = FALSE)
    acf_list[[param]] <- data.frame(
      Lag = as.numeric(acf_obj$lag),
      ACF = as.numeric(acf_obj$acf),
      Parameter = param
    )
  }
  acf_df <- do.call(rbind, acf_list)
  acf_df$Parameter <- factor(acf_df$Parameter,
                             levels = params_list
  )
  acf_df$Parameter <- factor(acf_df$Parameter,
                             levels = params_list,
                             labels = label_map[params_list]
  )
  acf_plot <- ggplot(acf_df, aes(x = Lag, y = ACF)) +
    geom_hline(yintercept = 0, color = "black", linewidth = 0.4) +
    geom_segment(aes(xend = Lag, yend = 0), color = "black", linewidth = 0.7) +
    facet_wrap(~Parameter, ncol = 1, labeller = label_parsed) +
    labs(x = "Lag", y = "ACF") +
    theme_classic(base_size = 12, base_family = "serif") +
    theme(
      panel.border = element_rect(fill = NA, color = "black", linewidth = 0.5),
      axis.line = element_blank(),
      strip.background = element_blank(),
      strip.text = element_text(face = "bold", size = 12, family = "serif"),
      legend.position = "none",
      legend.text = element_text(size = 9, family = "serif")
    )
  return(list(acf = acf_plot, trace = trace_plot, density = hist_plot))
}

# Linear Model Log-Posterior for BF comparison (via BridgeSampling)

log_posterior_linear <- function(samples.row, data) {
  T0 <- samples.row["T0"]
  T_dot <- samples.row["T_dot"]
  sigma <- samples.row["sigma"]

  # Log priors + Log likelihood
  dnorm(T0, 0, 20, log = TRUE) +
    dnorm(T_dot, 0, 20, log = TRUE) +
    dnorm(sigma, 0, 10, log = TRUE) - pnorm(0, 0, 10, lower.tail = FALSE, log.p = TRUE) +
    sum(dnorm(data$T_obs, T0 + T_dot * data$time_vec, sigma, log = TRUE))
```

```
}

# Constant Model Log-Posterior for BF comparison (via BridgeSampling)

log_posterior_constant <- function(samples.row, data) {
  T0 <- samples.row["T0"]
  sigma <- samples.row["sigma"]

  # Log priors + Log likelihood
  dnorm(T0, 0, 20, log = TRUE) +
    dnorm(sigma, 0, 10, log = TRUE) - pnorm(0, 0, 10, lower.tail = FALSE, log.p = TRUE) +
    sum(dnorm(data$T_obs, T0, sigma, log = TRUE))
}

# Predictive pdf functions

linear_predictive_pdf <- function(chain, t_new) {

  t0_samples    <- chain$T0
  t_dot_samples <- chain$T_dot
  sigma_samples <- chain$sigma

  mu_components    <- t0_samples + t_dot_samples * t_new
  sigma_components <- sigma_samples

  # Return PDF function
  function(x) {
    sapply(x, function(val) {
      mean(dnorm(val, mean = mu_components, sd = sigma_components))
    })
  }
}

constant_predictive_pdf <- function(chain, t_new) {

  t0_samples    <- chain$T0
  sigma_samples <- chain$sigma

  mu_components    <- t0_samples
  sigma_components <- sigma_samples

  # Return PDF function
  function(x) {
    sapply(x, function(val) {
      mean(dnorm(val, mean = mu_components, sd = sigma_components))
    })
  }
}

# Pred PDF plot function

plot_predictive_comparison <- function(draws_lin, draws_const, time_future,
                                       years_sd, years_mean,
                                       temp_range = c(5, 20)) {

  layout_matrix <- matrix(c(
    1, 2,  3,  4,  5,  6,
    1, 7,  8,  9, 10, 11
  ), nrow = 2, byrow = TRUE)

  layout(layout_matrix, widths = c(0.5, 1, 1, 1, 1, 1))

  par(mar = c(0, 0, 0, 0))

  plot(0, 0, type = "n", bty = "n",
       xaxt = "n", yaxt = "n",
       xlab = "", ylab = "")

  legend("center",
         legend = c("Linear", "Constant"),
```

```
        col = c("darkblue", "red"),
        lwd = 2,
        lty = c(1, 2),
        bty = "n",
        cex = 1.3,         # ↑ legend text size
        text.font = 2)

  temperature_grid <- seq(temp_range[1], temp_range[2], length.out = 300)
  years_future_real <- (time_future * years_sd) + years_mean

  n_steps <- length(time_future)

  for (i in 1:n_steps) {

    par(mar = c(4.5, 4.5, 3, 1))  # ↑ slightly larger margins for labels

    pdf_lin_fun <- linear_predictive_pdf(draws_lin, time_future[i])
    pdf_const_fun <- constant_predictive_pdf(draws_const, time_future[i])

    density_lin <- pdf_lin_fun(temperature_grid)
    density_const <- pdf_const_fun(temperature_grid)

    y_max <- max(c(density_lin, density_const)) * 1.15

    plot(temperature_grid, density_lin,
         type = "l",
         lwd = 2,
         col = "darkblue",
         xlab = "Temp (°C)",
         ylab = "Density",
         main = paste0("Step ", i, "\n(", round(years_future_real[i]), ")"),
         cex.main = 1.3,      # ↑ title
         cex.axis = 1.2,      # ↑ tick labels
         cex.lab = 1.3,       # ↑ axis labels
         ylim = c(0, y_max))

    lines(temperature_grid, density_const, lwd = 2, col = "red", lty = 2)

    grid()
  }

  layout(1)
}

# Predictive summaries

predictive_summaries <- function(chain, time_vector, model_type = "linear") {

  results_list <- lapply(time_vector, function(t_val) {

    if (model_type == "linear") {
      pdf_fun <- linear_predictive_pdf(chain, t_val)
      mu_chain <- chain$T0 + chain$T_dot * t_val
    } else {
      pdf_fun <- constant_predictive_pdf(chain, t_val)
      mu_chain <- chain$T0
    }

    exact_mean <- mean(mu_chain)
    exact_sd <- sqrt(mean(chain$sigma^2) + mean((mu_chain - exact_mean)^2)) # variance decomposition for gaussian mixture

    temp_grid <- seq(exact_mean - 6 * exact_sd, exact_mean + 6 * exact_sd, length.out = 1200)
    dx <- temp_grid[2] - temp_grid[1]

    densities <- pdf_fun(temp_grid)
    densities <- densities / sum(densities * dx) # numerical normalization of the mixture

    map_estimate <- temp_grid[which.max(densities)]

    cdf <- cumsum(densities * dx) # numerical approx of the cdf
```

```
    ci_lower <- temp_grid[which.min(abs(cdf - 0.025))]
    ci_upper <- temp_grid[which.min(abs(cdf - 0.975))]

    data.frame(
      Time_Step = t_val,
      MAP = round(map_estimate, 3),
      Mean = round(exact_mean, 3),
      SD = round(exact_sd, 3),
      CI_2.5 = round(ci_lower, 3),
      CI_97.5 = round(ci_upper, 3)
    )
  })

  return(do.call(rbind, results_list))
}
```

Execution (function calls etc.)

```
## ----setup, message=FALSE, warning=FALSE-----------------------------------
set.seed(123)
source("functions.R")


## --------------------------------------------------------------------------
data_ms <- find_data("data/Monte_Summano.csv")
data_ms



## ----warning=FALSE---------------------------------------------------------

month <- c("GEN", "FEB", "MAR", "APR", "MAG", "GIU",
                  "LUG", "AGO", "SET", "OTT", "NOV", "DIC", "Medio")

month_name <- c("January", "February", "March", "April", "May", "June", "July",
                "August", "September", "October", "November", "December", "Mean")

for (i in seq_along(month)) {
  m <- month[i]
  m_name <- month_name[i]

  temp_min_ms  <- data_ms[[1]][[m]]
  temp_mean_ms <- data_ms[[2]][[m]]
  temp_max_ms  <- data_ms[[3]][[m]]

  plot_hist(temp_min_ms, temp_mean_ms, temp_max_ms,
            month = m_name,
            city = "Monte Summano")
}


## ----fig.width=14, fig.height=18, dpi=300----------------------------------
plot_hist_all(data_ms, city = "Monte Summano")


## --------------------------------------------------------------------------
ms_means <- data_ms[[2]]
ms_means_yearavg <- ms_means$Medio

cat("Yearly means: \n")
cat(ms_means_yearavg, "\n")

ms_m_ya_length <- length(ms_means_yearavg)
cat("Number of yearly means: \n")
cat(ms_m_ya_length, "\n")

years_ms <- ms_means$Anno
cat("Years: \n")
```

```r
cat(years_ms, "\n")


## ------------------------------------------------------------------------
years_mean_ms <- mean(years_ms)
cat("Mean year:", years_mean_ms, "\n")

years_sd_ceiling_ms <- ceiling(sd(years_ms))
cat("Ceiling of standard deviation:", years_sd_ceiling_ms, "\n")

years_stdd_ms <- (years_ms - years_mean_ms) / years_sd_ceiling_ms

N_ms <- ms_m_ya_length
cat("Number of observations:", N_ms, "\n")

step_ms <- abs(years_stdd_ms[1] - years_stdd_ms[2])
cat("Standardized time step:", step_ms, "\n")

N_fut_steps_ms <- 10
cat("Number of future prediction steps:", N_fut_steps_ms, "\n")

last_observed_time_ms <- max(years_stdd_ms)
time_future_ms <- seq(last_observed_time_ms + step_ms, by = step_ms, length.out = N_fut_steps_ms)

cat("Future standardized time points:\n")
print(time_future_ms)

stan_data_ms <- list(
  N = N_ms,
  time = years_stdd_ms,
  T = ms_means_yearavg,
  N_future = N_fut_steps_ms,
  time_future = time_future_ms
)


## ----warning=TRUE, message=TRUE------------------------------------------------
res_ms <- build_fit_stan_model("./Stan/linear_model.stan", stan_data_ms)

mod_ms <- res_ms$mod
fit_ms <- res_ms$fit

stan_T_param_names <- c("T0", "T_dot")

cat("Summary of T parameters:\n")
fit_ms$summary(stan_T_param_names)

cat("Summary of sigma:\n")
fit_ms$summary("sigma")

cat("Running CmdStan diagnostics...\n")
fit_ms$cmdstan_diagnose()



## ----warning=FALSE-------------------------------------------------------------
draws_df_ms <- as_draws_df(fit_ms$draws())

quantiles_T_ms <- lapply(
  draws_df_ms[, stan_T_param_names],
  quantile,
  probs = c(0.025, 0.975)
)

cat("T0    : [",
    round(quantiles_T_ms$T0[1], 3), ", ",
    round(quantiles_T_ms$T0[2], 3), "]\n")

cat("T_dot : [",
```

```
    round(quantiles_T_ms$T_dot[1], 3), ", ",
    round(quantiles_T_ms$T_dot[2], 3), "]\n")
cat("\n")



## ------------------------------------------------------------------------
cor_plot_ms <- plot_2_correlations(draws_df_ms, stan_T_param_names, axes_names = c(expression(T[0]), expression(dot(T))))
print(cor_plot_ms$plot)
cor_plot_ms$cor

cor_plot_sigma_0_ms <- plot_2_correlations(draws_df_ms, c("T0", "sigma"), axes_names = c(expression(T[0]), expression(sigma)))
print(cor_plot_sigma_0_ms$plot)
cor_plot_sigma_0_ms$cor

cor_plot_sigma_dot_ms <- plot_2_correlations(draws_df_ms, c("T_dot", "sigma"),
                                             axes_names = c(expression(dot(T)), expression(sigma)))
print(cor_plot_sigma_dot_ms$plot)
cor_plot_sigma_dot_ms$cor



## ----fig.width=16, fig.height=5-----------------------------------------------
p1 <- cor_plot_ms$plot
p2 <- cor_plot_sigma_0_ms$plot
p3 <- cor_plot_sigma_dot_ms$plot

(p1 | p2 | p3)


## ------------------------------------------------------------------------
plot_ms <- plot_posterior(draws_df_ms,
                          my_x_stdd = stan_data_ms$time,
                          my_y_data = stan_data_ms$T,
                          n_grid = N_ms,
                          n_pred_grid = N_fut_steps_ms,
                          orig_mean = years_mean_ms,
                          orig_sd = years_sd_ceiling_ms,
                          my_step = step_ms)
print(plot_ms)


## ------------------------------------------------------------------------
graphs_ms <- plot_various_graphs(fit_ms, c("T0", "T_dot", "sigma"))

print(graphs_ms$trace)
print(graphs_ms$density)
print(graphs_ms$acf)



## ----warning=FALSE------------------------------------------------------------
res_c_ms <- build_fit_stan_model("./Stan/constant_model.stan", stan_data_ms)

mod_c_ms <- res_c_ms$mod
fit_c_ms <- res_c_ms$fit

cat("Summary of T, sigma parameters:\n")
fit_c_ms$summary(c("T0", "sigma"))

cat("Running CmdStan diagnostics...\n")
fit_c_ms$cmdstan_diagnose()



## ----warning=FALSE------------------------------------------------------------
draws_df_c_ms <- as_draws_df(fit_c_ms$draws())

quantiles_c_ms <- lapply(
  draws_df_c_ms[, c("T0", "sigma")],
```

```
  quantile,
  probs = c(0.025, 0.975)
)

cat("T0 : [",
    round(quantiles_c_ms$T0[1], 3), ", ",
    round(quantiles_c_ms$T0[2], 3), "]\n")

cat("sigma : [",
    round(quantiles_c_ms$sigma[1], 3), ", ",
    round(quantiles_c_ms$sigma[2], 3), "]\n")
cat("\n")



## ----warning=FALSE-----------------------------------------------------------
cor_plot_c_ms <- plot_2_correlations(draws_df_c_ms, c("T0", "sigma"), axes_names = c(expression(T[0]), expression(sigma)))
print(cor_plot_c_ms$plot)
cor_plot_c_ms$cor



## ----------------------------------------------------------------------------
plot_ms_c <- plot_posterior(draws_df_c_ms,
                            my_x_stdd = stan_data_ms$time,
                            my_y_data = stan_data_ms$T,
                            n_grid = N_ms,
                            n_pred_grid = N_fut_steps_ms,
                            orig_mean = years_mean_ms,
                            orig_sd = years_sd_ceiling_ms,
                            my_step = step_ms)
print(plot_ms_c)



## ----------------------------------------------------------------------------
c_graphs_ms <- plot_various_graphs(fit_c_ms, c("T0", "sigma"))
print(c_graphs_ms$trace)
print(c_graphs_ms$density)
print(c_graphs_ms$acf)



## ----------------------------------------------------------------------------
set.seed(123)
data_list_ms <- list(T_obs = stan_data_ms$T, time_vec = stan_data_ms$time)

log_marg_lik_linear_ms <- bridge_sampler(
  samples = fit_ms$draws(variables = c("T0", "T_dot", "sigma"), format = "matrix"),
  log_posterior = log_posterior_linear, data = data_list_ms,
  lb = c(T0 = -Inf, T_dot = -Inf, sigma = 0),
  ub = c(T0 = Inf, T_dot = Inf, sigma = Inf),
  method = "normal", silent = TRUE
)

log_marg_lik_constant_ms <- bridge_sampler(
  samples = fit_c_ms$draws(variables = c("T0", "sigma"), format = "matrix"),
  log_posterior = log_posterior_constant, data = data_list_ms,
  lb = c(T0 = -Inf, sigma = 0),
  ub = c(T0 = Inf, sigma = Inf),
  method = "normal", silent = TRUE
)

bf_ms <- bayes_factor(log_marg_lik_linear_ms, log_marg_lik_constant_ms)

cat("BAYES FACTOR MODEL COMPARISON\n\n")
cat("Log Marginal Likelihood (Linear Model):  ", round(log_marg_lik_linear_ms$logml, 3), "\n")
cat("Log Marginal Likelihood (Constant Model):", round(log_marg_lik_constant_ms$logml, 3), "\n")
cat("Bayes Factor (M_Lin / M_Const):          ", round(bf_ms$bf, 3), "\n")
```

```r
## ----warning=FALSE-------------------------------------------------------------
log_lik_ms <- fit_ms$draws("log_lik", format = "matrix")
loo_res_ms <- loo(log_lik_ms)

c_log_lik_ms <- fit_c_ms$draws("log_lik", format = "matrix")
c_loo_res_ms <- loo(c_log_lik_ms)

elpd_lin_i_ms <- loo_res_ms$pointwise[, "elpd_loo"]
elpd_c_i_ms   <- c_loo_res_ms$pointwise[, "elpd_loo"]

elpd_diff_i_ms <- elpd_lin_i_ms - elpd_c_i_ms

elpd_diff_ms <- sum(elpd_diff_i_ms)
se_diff_ms   <- sqrt(length(elpd_diff_i_ms) * var(elpd_diff_i_ms))

cat("LOO-CV MODEL COMPARISON (Montesummano)\n\n")
cat("ELPD difference (M_Lin - M_Const):", round(elpd_diff_ms, 3), "\n")
cat("Standard error:", round(se_diff_ms, 3), "\n")


## ----fig.width=12, fig.height=5, dpi=300---------------------------------------
(plot_ms_c | plot_ms )


## ----warning=FALSE, message=FALSE, fig.width=12, fig.height=6, echo=FALSE-----
plot_predictive_comparison(
  draws_lin = draws_df_ms,
  draws_const = draws_df_c_ms,
  time_future = time_future_ms,
  years_sd = years_sd_ceiling_ms,
  years_mean = years_mean_ms
)


## -----------------------------------------------------------------------------
summary_lin_ms <- predictive_summaries(
  chain = draws_df_ms,
  time_vector = time_future_ms,
  model_type = "linear"
)

summary_lin_ms$Year <- round((summary_lin_ms$Time_Step * years_sd_ceiling_ms) + years_mean_ms)
summary_lin_ms_view <- summary_lin_ms[, c("Year", "MAP", "Mean", "SD", "CI_2.5", "CI_97.5")]
print(round(summary_lin_ms_view, 3))


## -----------------------------------------------------------------------------
summary_c_ms <- predictive_summaries(
  chain = draws_df_c_ms,
  time_vector = time_future_ms,
  model_type = "constant"
)

summary_c_ms$Year <- round((summary_c_ms$Time_Step * years_sd_ceiling_ms) + years_mean_ms)
summary_c_ms_view <- summary_c_ms[, c("Year", "MAP", "Mean", "SD", "CI_2.5", "CI_97.5")]
print(round(summary_c_ms_view, 3))


## ----warning=FALSE, message=FALSE, error=FALSE---------------------------------
full_plot <- plot_ms +
  geom_point(
    data = summary_lin_ms,
    aes(x = Time_Step, y = MAP, color = "MAP Pred."),
    inherit.aes = FALSE, shape = 16, size = 1.5, stroke = 1.2
  ) +
  scale_color_manual(
    values = c(
      "Observations" = "black",
```

```r
      "Posterior mean" = "blue",
      "MAP Pred." = "red"
    )
  )

print(full_plot)


## ----warning=FALSE, message=FALSE, error=FALSE--------------------------------
full_plot <- plot_ms_c +
  geom_point(
    data = summary_c_ms,
    aes(x = Time_Step, y = MAP, color = "MAP Pred."),
    inherit.aes = FALSE, shape = 16, size = 1.5, stroke = 1.2
  ) +
  scale_color_manual(
    values = c(
      "Observations" = "black",
      "Posterior mean" = "blue",
      "MAP Pred." = "red"
    )
  )

print(full_plot)


## -----------------------------------------------------------------------------
data_od <- find_data("data/Oderzo.csv")
data_od <- lapply(data_od, function(df) df[-nrow(df), ]) # removed 2024 (only Jan meas)
data_od



## ----warning=FALSE------------------------------------------------------------

for (i in seq_along(month)) {
  m <- month[i]
  m_name <- month_name[i]

  temp_min_od  <- data_od[[1]][[m]]
  temp_mean_od <- data_od[[2]][[m]]
  temp_max_od  <- data_od[[3]][[m]]

  plot_hist(temp_min_od, temp_mean_od, temp_max_od,
            month = m_name,
            city = "Oderzo")
}


## ----fig.width=14, fig.height=18, dpi=300-------------------------------------
plot_hist_all(data_od, city = "Oderzo")


## -----------------------------------------------------------------------------
od_means <- data_od[[2]]
od_means_yearavg <- od_means$Medio

cat("Yearly means: \n")
cat(od_means_yearavg, "\n")

od_m_ya_length <- length(od_means_yearavg)
cat("Number of yearly means: \n")
cat(od_m_ya_length, "\n")

years_od <- od_means$Anno
cat("Years: \n")
cat(years_od, "\n")
```

```r
## ----------------------------------------------------------------------------
years_mean_od <- mean(years_od)
cat("Mean year:", years_mean_od, "\n")

years_sd_ceiling_od <- ceiling(sd(years_od))
cat("Ceiling of standard deviation:", years_sd_ceiling_od, "\n")

years_stdd_od <- (years_od - years_mean_od) / years_sd_ceiling_od

N_od <- od_m_ya_length
cat("Number of observations:", N_od, "\n")

step_od <- abs(years_stdd_od[1] - years_stdd_od[2])
cat("Standardized time step:", step_od, "\n")

N_fut_steps_od <- 10
cat("Number of future prediction steps:", N_fut_steps_od, "\n")

last_observed_time_od <- max(years_stdd_od)
time_future_od <- seq(last_observed_time_od + step_od, by = step_od, length.out = N_fut_steps_od)

cat("Future standardized time points:\n")
print(time_future_od)

stan_data_od <- list(
  N = N_od,
  time = years_stdd_od,
  T = od_means_yearavg,
  N_future = N_fut_steps_od,
  time_future = time_future_od
)



## ----warning=TRUE, message=TRUE----------------------------------------------
res_od <- build_fit_stan_model("./Stan/linear_model.stan", stan_data_od)

mod_od <- res_od$mod
fit_od <- res_od$fit

cat("Summary of T parameters:\n")
fit_od$summary(stan_T_param_names)

cat("Summary of sigma:\n")
fit_od$summary("sigma")

cat("Running CmdStan diagnostics...\n")
fit_od$cmdstan_diagnose()



## ----warning=FALSE-----------------------------------------------------------
draws_df_od <- as_draws_df(fit_od$draws())

quantiles_T_od <- lapply(
  draws_df_od[, stan_T_param_names],
  quantile,
  probs = c(0.025, 0.975)
)

cat("T0    : [",
    round(quantiles_T_od$T0[1], 3), ", ",
    round(quantiles_T_od$T0[2], 3), "]\n")

cat("T_dot : [",
    round(quantiles_T_od$T_dot[1], 3), ", ",
    round(quantiles_T_od$T_dot[2], 3), "]\n")
cat("\n")
```

```
## ----warning=FALSE-----------------------------------------------------------
cor_plot_od <- plot_2_correlations(draws_df_od, stan_T_param_names, axes_names = c(expression(T[0]), expression(dot(T))))
print(cor_plot_od$plot)
cor_plot_od$cor

cor_plot_sigma_0_od <- plot_2_correlations(draws_df_od, c("T0", "sigma"), axes_names = c(expression(T[0]), expression(sigma)))
print(cor_plot_sigma_0_od$plot)
cor_plot_sigma_0_od$cor

cor_plot_sigma_dot_od <- plot_2_correlations(draws_df_od, c("T_dot", "sigma"),
                                             axes_names = c(expression(dot(T)), expression(sigma)))
print(cor_plot_sigma_dot_od$plot)
cor_plot_sigma_dot_od$cor


## ----fig.width=16, fig.height=5----------------------------------------------
p1 <- cor_plot_od$plot
p2 <- cor_plot_sigma_0_od$plot
p3 <- cor_plot_sigma_dot_od$plot

(p1 | p2 | p3)


## ----------------------------------------------------------------------------
plot_od <- plot_posterior(draws_df_od,
                          my_x_stdd = stan_data_od$time,
                          my_y_data = stan_data_od$T,
                          n_grid = N_od,
                          n_pred_grid = N_fut_steps_od,
                          orig_mean = years_mean_od,
                          orig_sd = years_sd_ceiling_od,
                          my_step = step_od)
print(plot_od)


## ----------------------------------------------------------------------------
graphs_od <- plot_various_graphs(fit_od, c("T0", "T_dot", "sigma"))

print(graphs_od$trace)
print(graphs_od$density)
print(graphs_od$acf)



## ----warning=FALSE-----------------------------------------------------------
res_c_od <- build_fit_stan_model("./Stan/constant_model.stan", stan_data_od)

mod_c_od <- res_c_od$mod
fit_c_od <- res_c_od$fit

cat("Summary of T, sigma parameters:\n")
fit_c_od$summary(c("T0", "sigma"))

cat("Running CmdStan diagnostics...\n")
fit_c_od$cmdstan_diagnose()



## ----warning=FALSE-----------------------------------------------------------
draws_df_c_od <- as_draws_df(fit_c_od$draws())

quantiles_c_od <- lapply(
  draws_df_c_od[, c("T0", "sigma")],
  quantile,
  probs = c(0.025, 0.975)
)

cat("T0 : [",
    round(quantiles_c_od$T0[1], 3), ", ",
```

```
    round(quantiles_c_od$T0[2], 3), "]\n")

cat("sigma : [",
    round(quantiles_c_od$sigma[1], 3), ", ",
    round(quantiles_c_od$sigma[2], 3), "]\n")
cat("\n")



## ----warning=FALSE-----------------------------------------------------------
cor_plot_c_od <- plot_2_correlations(draws_df_c_od, c("T0", "sigma"), axes_names = c(expression(T[0]), expression(sigma)))
print(cor_plot_c_od$plot)
cor_plot_c_od$cor



## ----------------------------------------------------------------------------
plot_od_c <- plot_posterior(draws_df_c_od,
                            my_x_stdd = stan_data_od$time,
                            my_y_data = stan_data_od$T,
                            n_grid = N_od,
                            n_pred_grid = N_fut_steps_od,
                            orig_mean = years_mean_od,
                            orig_sd = years_sd_ceiling_od,
                            my_step = step_od)
print(plot_od_c)


## ----------------------------------------------------------------------------
c_graphs_od <- plot_various_graphs(fit_c_od, c("T0", "sigma"))
print(c_graphs_od$trace)
print(c_graphs_od$density)
print(c_graphs_od$acf)



## ----------------------------------------------------------------------------
set.seed(123)
data_list_od <- list(T_obs = stan_data_od$T, time_vec = stan_data_od$time)

log_marg_lik_linear_od <- bridge_sampler(
  samples = fit_od$draws(variables = c("T0", "T_dot", "sigma"), format = "matrix"),
  log_posterior = log_posterior_linear, data = data_list_od,
  lb = c(T0 = -Inf, T_dot = -Inf, sigma = 0),
  ub = c(T0 = Inf, T_dot = Inf, sigma = Inf),
  method = "normal", silent = TRUE
)

log_marg_lik_constant_od <- bridge_sampler(
  samples = fit_c_od$draws(variables = c("T0", "sigma"), format = "matrix"),
  log_posterior = log_posterior_constant, data = data_list_od,
  lb = c(T0 = -Inf, sigma = 0),
  ub = c(T0 = Inf, sigma = Inf),
  method = "normal", silent = TRUE
)

bf_od <- bayes_factor(log_marg_lik_linear_od, log_marg_lik_constant_od)

cat("BAYES FACTOR MODEL COMPARISON\n\n")
cat("Log Marginal Likelihood (Linear Model):  ", round(log_marg_lik_linear_od$logml, 3), "\n")
cat("Log Marginal Likelihood (Constant Model):", round(log_marg_lik_constant_od$logml, 3), "\n")
cat("Bayes Factor (M_Lin / M_Const):          ", round(bf_od$bf, 3), "\n")



## ----warning=FALSE-----------------------------------------------------------
log_lik_od <- fit_od$draws("log_lik", format = "matrix")
loo_res_od <- loo(log_lik_od)

c_log_lik_od <- fit_c_od$draws("log_lik", format = "matrix")
```

```r
c_loo_res_od <- loo(c_log_lik_od)

elpd_lin_i_od <- loo_res_od$pointwise[, "elpd_loo"]
elpd_c_i_od   <- c_loo_res_od$pointwise[, "elpd_loo"]

elpd_diff_i_od <- elpd_lin_i_od - elpd_c_i_od

elpd_diff_od <- sum(elpd_diff_i_od)
se_diff_od   <- sqrt(length(elpd_diff_i_od) * var(elpd_diff_i_od))

cat("LOO-CV MODEL COMPARISON (Oderzo)\n\n")
cat("ELPD difference (M_Lin - M_Const):", round(elpd_diff_od, 3), "\n")
cat("Standard error:", round(se_diff_od, 3), "\n")



## ----fig.width=12, fig.height=5, dpi=300-------------------------------------
(plot_od_c | plot_od )


## ----warning=FALSE, message=FALSE, fig.width=12, fig.height=6, echo=FALSE-----
plot_predictive_comparison(
  draws_lin = draws_df_od,
  draws_const = draws_df_c_od,
  time_future = time_future_od,
  years_sd = years_sd_ceiling_od,
  years_mean = years_mean_od
)


## ----------------------------------------------------------------------------
summary_lin_od <- predictive_summaries(
  chain = draws_df_od,
  time_vector = time_future_od,
  model_type = "linear"
)

summary_lin_od$Year <- round((summary_lin_od$Time_Step * years_sd_ceiling_od) + years_mean_od)
summary_lin_od_view <- summary_lin_od[, c("Year", "MAP", "Mean", "SD", "CI_2.5", "CI_97.5")]
print(round(summary_lin_od_view, 3))


## ----------------------------------------------------------------------------
summary_c_od <- predictive_summaries(
  chain = draws_df_c_od,
  time_vector = time_future_od,
  model_type = "constant"
)

summary_c_od$Year <- round((summary_c_od$Time_Step * years_sd_ceiling_od) + years_mean_od)
summary_c_od_view <- summary_c_od[, c("Year", "MAP", "Mean", "SD", "CI_2.5", "CI_97.5")]
print(round(summary_c_od_view, 3))


## ----warning=FALSE, message=FALSE, error=FALSE-------------------------------
full_plot <- plot_od +
  geom_point(
    data = summary_lin_od,
    aes(x = Time_Step, y = MAP, color = "MAP Pred."),
    inherit.aes = FALSE, shape = 16, size = 1.5, stroke = 1.2
  ) +
  scale_color_manual(
    values = c(
      "Observations" = "black",
      "Posterior mean" = "blue",
      "MAP Pred." = "red"
    )
  )

print(full_plot)
```

```r
## ----warning=FALSE, message=FALSE, error=FALSE--------------------------------
full_plot <- plot_od_c +
  geom_point(
    data = summary_c_od,
    aes(x = Time_Step, y = MAP, color = "MAP Pred."),
    inherit.aes = FALSE, shape = 16, size = 1.5, stroke = 1.2
  ) +
  scale_color_manual(
    values = c(
      "Observations" = "black",
      "Posterior mean" = "blue",
      "MAP Pred." = "red"
    )
  )

print(full_plot)


## -----------------------------------------------------------------------------
data_ci <- find_data("data/Cittadella.csv")
data_ci



## ----warning=FALSE------------------------------------------------------------

for (i in seq_along(month)) {
  m <- month[i]
  m_name <- month_name[i]

  temp_min_ci  <- data_ci[[1]][[m]]
  temp_mean_ci <- data_ci[[2]][[m]]
  temp_max_ci  <- data_ci[[3]][[m]]

  plot_hist(temp_min_ci, temp_mean_ci, temp_max_ci,
            month = m_name,
            city = "Cittadella")
}


## ----fig.width=14, fig.height=18, dpi=300-------------------------------------
plot_hist_all(data_ci, city = "Cittadella")


## -----------------------------------------------------------------------------
ci_means <- data_ci[[2]]
ci_means_yearavg <- ci_means$Medio

cat("Yearly means: \n")
cat(ci_means_yearavg, "\n")

ci_m_ya_length <- length(ci_means_yearavg)
cat("Number of yearly means: \n")
cat(ci_m_ya_length, "\n")

years_ci <- ci_means$Anno
cat("Years: \n")
cat(years_ci, "\n")



## -----------------------------------------------------------------------------
years_mean_ci <- mean(years_ci)
cat("Mean year:", years_mean_ci, "\n")

years_sd_ceiling_ci <- ceiling(sd(years_ci))
cat("Ceiling of standard deviation:", years_sd_ceiling_ci, "\n")
```

```r
years_stdd_ci <- (years_ci - years_mean_ci) / years_sd_ceiling_ci

N_ci <- ci_m_ya_length
cat("Number of observations:", N_ci, "\n")

step_ci <- abs(years_stdd_ci[1] - years_stdd_ci[2])
cat("Standardized time step:", step_ci, "\n")

N_fut_steps_ci <- 10
cat("Number of future prediction steps:", N_fut_steps_ci, "\n")

last_observed_time_ci <- max(years_stdd_ci)
time_future_ci <- seq(last_observed_time_ci + step_ms, by = step_ci, length.out = N_fut_steps_ci)

cat("Future standardized time points:\n")
print(time_future_ci)

stan_data_ci <- list(
  N = N_ci,
  time = years_stdd_ci,
  T = ci_means_yearavg,
  N_future = N_fut_steps_ci,
  time_future = time_future_ci
)



## ----warning=TRUE, message=TRUE-----------------------------------------------
res_ci <- build_fit_stan_model("./Stan/linear_model.stan", stan_data_ci)

mod_ci <- res_ci$mod
fit_ci <- res_ci$fit

cat("Summary of T parameters:\n")
fit_ci$summary(stan_T_param_names)

cat("Summary of sigma:\n")
fit_ci$summary("sigma")

cat("Running CmdStan diagnostics...\n")
fit_ci$cmdstan_diagnose()



## ----warning=FALSE------------------------------------------------------------
draws_df_ci <- as_draws_df(fit_ci$draws())

quantiles_T_ci <- lapply(
  draws_df_ci[, stan_T_param_names],
  quantile,
  probs = c(0.025, 0.975)
)

cat("T0    : [",
    round(quantiles_T_ci$T0[1], 3), ", ",
    round(quantiles_T_ci$T0[2], 3), "]\n")

cat("T_dot : [",
    round(quantiles_T_ci$T_dot[1], 3), ", ",
    round(quantiles_T_ci$T_dot[2], 3), "]\n")
cat("\n")



## ----warning=FALSE------------------------------------------------------------
cor_plot_ci <- plot_2_correlations(draws_df_ci, stan_T_param_names, axes_names = c(expression(T[0]), expression(dot(T))))
print(cor_plot_ci$plot)
cor_plot_ci$cor

cor_plot_sigma_0_ci <- plot_2_correlations(draws_df_ci, c("T0", "sigma"), axes_names = c(expression(T[0]), expression(sigma)))
```

```r
print(cor_plot_sigma_0_ci$plot)
cor_plot_sigma_0_ci$cor

cor_plot_sigma_dot_ci <- plot_2_correlations(draws_df_ci, c("T_dot", "sigma"),
                                            axes_names = c(expression(dot(T)), expression(sigma)))
print(cor_plot_sigma_dot_ci$plot)
cor_plot_sigma_dot_ci$cor


## ----fig.width=16, fig.height=5-----------------------------------------------
p1 <- cor_plot_ci$plot
p2 <- cor_plot_sigma_0_ci$plot
p3 <- cor_plot_sigma_dot_ci$plot

(p1 | p2 | p3)


## -----------------------------------------------------------------------------
plot_ci <- plot_posterior(draws_df_ci,
                          my_x_stdd = stan_data_ci$time,
                          my_y_data = stan_data_ci$T,
                          n_grid = N_ci,
                          n_pred_grid = N_fut_steps_ci,
                          orig_mean = years_mean_ci,
                          orig_sd = years_sd_ceiling_ci,
                          my_step = step_ci)
print(plot_ci)



## -----------------------------------------------------------------------------
graphs_ci <- plot_various_graphs(fit_ci, c("T0", "T_dot", "sigma"))

print(graphs_ci$trace)
print(graphs_ci$density)
print(graphs_ci$acf)



## ----warning=FALSE------------------------------------------------------------
res_c_ci <- build_fit_stan_model("./Stan/constant_model.stan", stan_data_ci)

mod_c_ci <- res_c_ci$mod
fit_c_ci <- res_c_ci$fit

cat("Summary of T, sigma parameters:\n")
fit_c_ci$summary(c("T0", "sigma"))

cat("Running CmdStan diagnostics...\n")
fit_c_ci$cmdstan_diagnose()



## ----warning=FALSE------------------------------------------------------------
draws_df_c_ci <- as_draws_df(fit_c_ci$draws())

quantiles_c_ci <- lapply(
  draws_df_c_ci[, c("T0", "sigma")],
  quantile,
  probs = c(0.025, 0.975)
)

cat("T0 : [",
    round(quantiles_c_ci$T0[1], 3), ", ",
    round(quantiles_c_ci$T0[2], 3), "]\n")

cat("sigma : [",
    round(quantiles_c_ci$sigma[1], 3), ", ",
    round(quantiles_c_ci$sigma[2], 3), "]\n")
cat("\n")
```

```r
## ----warning=FALSE-------------------------------------------------------
cor_plot_c_ci <- plot_2_correlations(draws_df_c_ci, c("T0", "sigma"), axes_names = c(expression(T[0]), expression(sigma)))
print(cor_plot_c_ci$plot)
cor_plot_c_ci$cor



## ------------------------------------------------------------------------
plot_ci_c <- plot_posterior(draws_df_c_ci,
                            my_x_stdd = stan_data_ci$time,
                            my_y_data = stan_data_ci$T,
                            n_grid = N_ci,
                            n_pred_grid = N_fut_steps_ci,
                            orig_mean = years_mean_ci,
                            orig_sd = years_sd_ceiling_ci,
                            my_step = step_ci)
print(plot_ci_c)



## ------------------------------------------------------------------------
c_graphs_ci <- plot_various_graphs(fit_c_ci, c("T0", "sigma"))
print(c_graphs_ci$trace)
print(c_graphs_ci$density)
print(c_graphs_ci$acf)



## ------------------------------------------------------------------------
set.seed(123)
data_list_ci <- list(T_obs = stan_data_ci$T, time_vec = stan_data_ci$time)

log_marg_lik_linear_ci <- bridge_sampler(
  samples = fit_ci$draws(variables = c("T0", "T_dot", "sigma"), format = "matrix"),
  log_posterior = log_posterior_linear, data = data_list_ci,
  lb = c(T0 = -Inf, T_dot = -Inf, sigma = 0),
  ub = c(T0 = Inf, T_dot = Inf, sigma = Inf),
  method = "normal", silent = TRUE
)

log_marg_lik_constant_ci <- bridge_sampler(
  samples = fit_c_ci$draws(variables = c("T0", "sigma"), format = "matrix"),
  log_posterior = log_posterior_constant, data = data_list_ci,
  lb = c(T0 = -Inf, sigma = 0),
  ub = c(T0 = Inf, sigma = Inf),
  method = "normal", silent = TRUE
)

bf_ci <- bayes_factor(log_marg_lik_linear_ci, log_marg_lik_constant_ci)

cat("BAYES FACTOR MODEL COMPARISON\n\n")
cat("Log Marginal Likelihood (Linear Model):  ", round(log_marg_lik_linear_ci$logml, 3), "\n")
cat("Log Marginal Likelihood (Constant Model):", round(log_marg_lik_constant_ci$logml, 3), "\n")
cat("Bayes Factor (M_Lin / M_Const):          ", round(bf_ci$bf, 3), "\n")



## ----warning=FALSE-------------------------------------------------------
log_lik_ci <- fit_ci$draws("log_lik", format = "matrix")
loo_res_ci <- loo(log_lik_ci)

c_log_lik_ci <- fit_c_ci$draws("log_lik", format = "matrix")
c_loo_res_ci <- loo(c_log_lik_ci)

elpd_lin_i_ci <- loo_res_ci$pointwise[, "elpd_loo"]
elpd_c_i_ci   <- c_loo_res_ci$pointwise[, "elpd_loo"]
```

```r
elpd_diff_i_ci <- elpd_lin_i_ci - elpd_c_i_ci

elpd_diff_ci   <- sum(elpd_diff_i_ci)
se_diff_ci     <- sqrt(length(elpd_diff_i_ci) * var(elpd_diff_i_ci))

cat("LOO-CV MODEL COMPARISON\n\n")
cat("ELPD difference (M_Lin - M_Const):", round(elpd_diff_ci, 3), "\n")
cat("Standard error:", round(se_diff_ci, 3), "\n")


## ----fig.width=12, fig.height=5, dpi=300-------------------------------------
(plot_ci_c | plot_ci )


## ----warning=FALSE, message=FALSE, fig.width=12, fig.height=6, echo=FALSE-----
plot_predictive_comparison(
  draws_lin = draws_df_ci,
  draws_const = draws_df_c_ci,
  time_future = time_future_ci,
  years_sd = years_sd_ceiling_ci,
  years_mean = years_mean_ci
)


## ----------------------------------------------------------------------------
summary_lin_ci <- predictive_summaries(
  chain = draws_df_ci,
  time_vector = time_future_ci,
  model_type = "linear"
)

summary_lin_ci$Year <- round((summary_lin_ci$Time_Step * years_sd_ceiling_ci) + years_mean_ci)
summary_lin_ci_view <- summary_lin_ci[, c("Year", "MAP", "Mean", "SD", "CI_2.5", "CI_97.5")]
print(round(summary_lin_ci_view, 3))


## ----------------------------------------------------------------------------
summary_c_ci <- predictive_summaries(
  chain = draws_df_c_ci,
  time_vector = time_future_ci,
  model_type = "constant"
)

summary_c_ci$Year <- round((summary_c_ci$Time_Step * years_sd_ceiling_ci) + years_mean_ci)
summary_c_ci_view <- summary_c_ci[, c("Year", "MAP", "Mean", "SD", "CI_2.5", "CI_97.5")]
print(round(summary_c_ci_view, 3))


## ----warning=FALSE, message=FALSE, error=FALSE-------------------------------
full_plot <- plot_ci +
  geom_point(
    data = summary_lin_ci,
    aes(x = Time_Step, y = MAP, color = "MAP Pred."),
    inherit.aes = FALSE, shape = 16, size = 1.5, stroke = 1.2
  ) +
  scale_color_manual(
    values = c(
      "Observations" = "black",
      "Posterior mean" = "blue",
      "MAP Pred." = "red"
    )
  )

print(full_plot)


## ----warning=FALSE, message=FALSE, error=FALSE-------------------------------
full_plot <- plot_ci_c +
  geom_point(
    data = summary_c_ci,
```

```
    aes(x = Time_Step, y = MAP, color = "MAP Pred."),
    inherit.aes = FALSE, shape = 16, size = 1.5, stroke = 1.2
  ) +
  scale_color_manual(
    values = c(
      "Observations" = "black",
      "Posterior mean" = "blue",
      "MAP Pred." = "red"
    )
  )

print(full_plot)
```

Declaration of AI use: AI assistance was used solely for improving the appearance of plots and for improving report readability (no code generation or analysis).